Universidad
Complutense & Politécnica
de Madrid

Master's in Formal Methods in Computer Science

Master's Final Project

Type Theory and Categorical Models
A Unified Approach

Miguel Laredo Barbadillo
Raúl Lara Cabrera
Guillermo Román Díez

Madrid, September 11, 2025

This Final Degree Project has been submitted to the ETSI Informáticos of the Universidad Politécnica de Madrid for its defense.

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Master's in	Formal Methods in Computer Science
Title	Type Theory and Categorical Models: A Unified Approach

Author	Miguel Laredo Barbadillo
Tutors	Raúl Lara Cabrera
	Dpto. Sistemas Informáticos
	Guillermo Román Díez
	Dpto. Lenguajes y Sistemas Informáticos e Ingeniería de Softwaree

Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

A los «Informales», que han hecho de este máster una experiencia inolvidable y llena de momentazos.
A Raúl Lara, porque trabajar con él es un gusto y un privilegio.
A mi familia, por su apoyo inagotable.

—Gracias.

Contents

Summary . . . 9

1 Untyped λ-Calculus **13**

The Intuitive Notion . . . 13

1.1 The Language of λ-Calculus . . . 13

1.2 Reduction of terms . . . 14

Subterms . . . 14

Free Variables . . . 15

Variable Substitution . . . 15

α-equivalence . . . 17

β-equivalence . . . 17

η-equivalence . . . 18

$\beta\eta$-reduction . . . 19

1.3 Normalization & The Church-Rosser Theorem . . . 19

The Church-Rosser in the λ-Calculus . . . 19

Proof Sketch of the Church-Rosser for $\beta\eta$-Reduction . . . 20

Normalization . . . 21

1.4 Encodings & Constructions in the λ-Calculus . . . 22

Natural Numbers . . . 22

Basic Arithmetic . . . 23

Booleans & Branching . . . 23

Pairs . . . 24

Lists . . . 24

1.5 Fixed Point Combinators and Recursion . . . 24

Fixed Points . . . 25

Recursive Combinators . . . 25

λ-Calculus Implementation of Factorial . . . 26

2 Typing **29**

2.1 Minimal Typing . . . 29

Typing à la Church . . . 30

Typing à la Curry . . . 31

Properties of Minimal Typing . . . 32

2.2 Extended Typing–Products . . . 33

Reduction Rules . . . 33

β-reduction . . . 33

η-laws . . . 34

$\beta\eta$-conversion . . . 34

2.3 Church Rosser Revisited . . . 34

2.4 Normalization Revisited . . . 34

Proof Sketch of the Weak Normalization Theorem . . . 35

2.5 Extended Typing–Sums . . . 36

Reduction Rules . . . 36

β-reduction . . . 36

η-laws . . . 36

$\beta\eta$-conversion . . . 37

2.6 Extended Typing–Sums and Products . . . 37

Kinds of problem in Type Theory . . . 37

3 Intuitionistic Logic & Curry–Howard **39**
3.1 Intuitionistic Logic . . . 39
Syntax . . . 39
The Brouwer–Heyting–Kolmogorov Interpretation . . . 40
Natural Deduction . . . 40
Introduction / Elimination Rules . . . 41
3.2 The Curry-Howard Isomorphism . . . 41

4 Categories at Work **43**
4.1 What are Categories? . . . 43
Hom-set . . . 44
Duality . . . 45
4.2 Monos & Epis & Isos . . . 46
4.3 Functors . . . 47
Contravariant Functors . . . 49
4.4 Natural Transformations . . . 49
4.5 Universal Constructions . . . 52
Terminal & Initial Objects . . . 53
Products and Coproducts . . . 53
Exponentials . . . 54
4.6 Closed Cartesian Categories . . . 55
Adjunctions . . . 55

5 Curry–Howard–Lambek & Categorical Semantics **57**
5.1 Categories as Deduction Systems . . . 57
5.2 Closing Remarks and Perspectives . . . 58
Dependent Typing . . . 58
Dependent Types and Categories . . . 58

Abstract

This work investigates the interplay of λ-calculus, type theory, category theory, through the Curry-Howard-Lambek correspondence. It explores λ-calculus as a computational foundation, its type-safe expression, and its categorical semantics. It focuses on why the λ-calculus is a suitable computation model as well as how does it relate to categories, and how this relation leads to advancements in programming languages and further topics.

It also develops some basic programs to evaluate λ-expressions, as well as different methods to perform inference and type checking on said expressions. These programs have an illustrative purpose rather than a practical one.

Este trabajo estudia la relación entre λ-cálculo, teoría de tipos y teoría de categorías a través de la correspondencia de Curry–Howard–Lambek. Se analiza el λ-cálculo como fundamento computacional, su formulación con tipado seguro y su semántica categórica. El estudio se centra en por qué el λ-cálculo constituye un modelo de computación adecuado, cómo se vincula con las categorías y de qué manera esta relación impulsa avances en los lenguajes de programación y en otros campos afines.

Además, se presentan algunos programas básicos para evaluar expresiones λ, junto con distintos métodos para realizar inferencia y verificación de tipos sobre ellas. Estos programas tienen un propósito ilustrativo más que práctico.

Keywords: Lambda Calculus, Type Theory, Category Theory, Curry-Howard-Lambek Correspondence.

Introduction

This work means to provide an introduction to functional programming and categorical reasoning about programs, using the λ-calculus as its foundational formalism for the study of computation. With this in mind, we cover confluence and normalization properties to an acceptable level of detail, as to provide the reader with enough evidence that this system is solid enough.

In its untyped form, the λ-calculus lacks the structure necessary to guarantee properties such as safeness and termination. This lack of structure, naturally leads into typing. We cover monomorphic type systems and the notion of *"programs as proofs"* as introduced by Howard 1980. Typing disciplines enrich the λ-calculus by imposing constraints on expressions, ensuring that only well-formed programs can be constructed, guaranteeing termination and safety. Type systems also give compositional structure to programs, such that given $g : A \to B, f : B \to C$ we can construct $f \circ g : A \to C$. The meaning of this composition is entirely determined by the meanings of f and g and the rule of composition. Compositionality shifts the focus from specific objects to the morphisms between them, and is arguably the core idea behind functional programming.

This principle—where the whole is defined by its parts and their composition—extends beyond functions between sets. Category theory abstracts this idea: it generalizes functions to morphisms and provides a unified language for compositional structure. The work by Lambek 1980 motivates the final chapters of this work, presenting Closed Cartesian Categories, and how they extend the Curry–Howard Isomorphism, giving a categorical interpretation of the work by Curry and Howard.

Summary

Categorical semantics provides a formal, structural interpretation of type systems using the language of category theory. The simply typed λ-calculus is a core formalism for defining functions with explicitly assigned types. Its syntax is inductively defined by types and terms.

Definition 0.0.1 (Types)**.** Types are defined recursively as follows:

1. Base types: $\alpha, \beta, \ldots$
2. Function types: If A and B are types, then $A \to B$ is a type.

Definition 0.0.2 (Terms)**.** Terms are defined recursively as follows:

1. Variables $x : A$
2. Constants
3. Function application $M\,N$ (if $M : A \to B$ and $N : A$, then $M\,N : B$)
4. Function abstraction $\lambda x.M : A \to B$ (if $M : B$ under assumption $x : A$)

Example 0.0.1 (Simple Function)**.** Let Nat and Bool be base types. Then

$$\lambda x : \mathrm{Nat}.\,\mathrm{isEven}(x) : \mathrm{Nat} \to \mathrm{Bool}$$

defines a function that maps natural numbers to booleans.

The simply typed λ-calculus can be interpreted as a **deduction system**. Types correspond to propositions, and terms correspond to proofs. Identity terms $\lambda x.x : A \to A$ represent trivial or reflexive proofs, while composition of terms models chaining of derivations. If $f : A \to B$ and $g : B \to C$, then

$$g \circ f := \lambda x.\, g(f\,x) : A \to C$$

represents the derived proof of C from A. This compositional structure mirrors the behavior of formal derivations in a logical system.

Definition 0.0.3 (Curry–Howard Correspondence)**.** The Curry–Howard correspondence establishes a structural analogy between typed λ-calculus and intuitionistic logic:

1. Function types $A \to B$ correspond to logical implication $A \Rightarrow B$
2. Terms correspond to constructive proofs

Example 0.0.2 (Identity Proof)**.** The identity term $\lambda x.x : A \to A$ corresponds to a trivial proof that A implies A.

Categories generalize this notion of deduction.

Definition 0.0.4 (Category)**.** A category $\mathscr{C}$ consists of:

1. Objects
2. Morphisms between objects
3. Identity morphisms
4. Composition of morphisms satisfying associativity and unit laws

Definition 0.0.5 (Categories as Deduction Systems)**.** Objects represent propositions or types, morphisms represent proofs, identity morphisms represent reflexive derivations, and composition represents chaining of proofs.

Exponentials in a category capture the notion of function types: for objects A and B, the exponential B^A represents morphisms from A to B. A morphism $f : X \to B^A$ is equivalent to a morphism $g : X \times A \to B$, encoding currying and abstraction.

Definition 0.0.6 (Curry–Howard–Lambek Correspondence)**.** Simply typed λ-calculi can be interpreted in Cartesian Closed Categories (CCCs), where:

1. Objects model types or propositions
2. Morphisms model proofs or terms
3. Composition mirrors sequential derivation

This correspondence provides a **compositional, structural semantics** for type systems, connecting logic, computation, and category theory. It allows reasoning about entire type systems through categorical structure without reference to specific terms, emphasizing the role of composition and abstraction in both logic and computation.

Contents. As already discussed, this work explores the untyped and typed lambda calculus and the Curry–Howard Isomorphism. We then expand the correspondence by adding categorical semantics to build what is referred to as the Curry–Howard–Lambek Correspondence.

- The first chapter is an introduction to the λ-calculus. It emphasizes confluence as a pillar for computation, it also introduces normalization as a desirable property.
- In the second chapter, typing is introduced, together with its so desired normalization properties. Also, differences with the untyped λ-calculus are discussed.
- The third chapter glances over intuitionistic logic and natural deduction. It introduces the Curry–Howard correspondence.
- The fifth chapter is about category theory and its constructions. We cover functors, natural transformations and adjunctions. We also cover the necessary universal constructions to build Closed Cartesian Categories, namely, terminal objects, products and exponentials.
- The last chapter extends the Curry–Howard correspondence, establishing a relation between λ-calculus and Closed Cartesian Categories.

Target Audience. This text is intended for computer scientists who may be unfamiliar with category theory but are interested in understanding the theoretical foundations of functional programming. It aims to provide readers with enough background on typing so that they can better understand the relation functional programming languages have with category theory, compositionality, type systems and other concepts inside of Category Theory that drive the development of sate-of-the-art programming.

This text assumes basic knowledge of mathematics, familiarity with programming, and a working understanding of recursion. No prior experience with functional programming or category theory is required, though some familiarity would not hurt.

Rationale behind our Approach. This text occupies a middle ground between works like Milewski 2019 and Lane 1971. The former emphasizes intuitiveness, and while it serves as a powerful toolkit for programmers, it gives little insight into the formalisms of categorical reasoning. The latter, presents category theory in full generality, making it inaccessible to non-mathematicians. Our approach introduces the fundamental notions in a somewhat mathematically precise manner first to then develop programmer-friendly examples, applications, and intuitions. This text is of course not in the same league as the aforementioned, it is a short and concise introduction and should be read as such.

External Resources. The code is meant as a learning aid, nevertheless, it can be given purpose. Similarity to the formal definitions is the driving design principle behind most of the code, as well as ease of use. We use the various programs to simplify some of the examples and avoid including unnecessarily large reduction procedures.

- To clone the code used to implement the λ-calculus:

  ```
  $ git clone https://github.com/laredo02/lcalc.git
  ```

  There are a few different files in the directory, each of which encapsulates a different approach to typing:

  - Untyped
  - Fully annotated typing
  - Typed with annotations only on abstractions i.e. Typing à la Church
  - Typed with monomorphic type inference i.e. Typing à la Curry

  Files with miscellaneous examples related to categories are also included, in particular those about functors, natural transformations and their composition may be of interest. Implementations are supposed to especially help programmers to digest some of the contents of this work. To use the provided tools, we recommend running them on `ghci`, Haskell's interactive shell.

- To clone the LaTeX source used to generate this document:

  ```
  $ git clone https://github.com/laredo02/ttcmua.git
  ```

Disclosure. In the development of this work, I have employed AI tools to assist me with typesetting and at times to look for synonyms, as English is not my native tongue. All the content, including explanations, examples, and discussions, is original and developed by me. Whenever external material has been incorporated, it has been cited appropriately. At all times, I have verified the correctness of the information up to my level of competence, ensuring that the final work reflects my own understanding and effort, which is the ultimate goal of the document.

Chapter 1

Untyped λ-Calculus

In the early 20th century, mathematicians such as Bertrand Russell, David Hilbert, and Kurt Gödel were trying to set complete, consistent, and decidable foundations for mathematics. Within this context, Alonzo Church, in his effort to formalize the notion of computability, introduced a minimal symbolic language based upon function abstraction and function application, this system is now called λ-calculus.

In 1936, Church used lambda calculus to address Hilberts Entscheidungsproblem –whether a mechanical method could determine the truth of any first-order logic statement. He proved no such algorithm exists, establishing the undecidability of first-order logic.

One of the nuances formalized by the λ-calculus is the distinction between extensional and intensional equality. The extensional approach to equivalence states that two functions are equivalent if they share input output pairs for every possible input. Its intensional counterpart extends this notion of equality by requiring that the procedures that compute these pairs share complexity i.e. they take the same steps toward yielding a result.[1]

The Lambda Calculus together with Type Theory lay the theoretical groundwork that shaped the design of languages like ML or the more recent Haskell. By itself, the untyped λ-calculus modeled the first versions of Lisp. More advanced type systems than those present in general purpose programming languages enable computers to verify mathematical statements and produce rigorous proofs.

The Intuitive Notion

In order to get acquainted with the λ-calculus, let us develop a simple example to familiarize ourselves before we begin with a more formal approach to this discipline. Consider the function $f(x) = x+1$, the most straightforward way to express this using λ-calculus would be $(\lambda x.x+1)$, where the lambda denotes that x is being captured and used as a parameter to perform some computation. Evaluating $f(2)$ using this newly created lambda term would look like this: $(\lambda x.x+1)(2) \rightsquigarrow (2+1) \rightsquigarrow 3$. This process of reducing a λ-expression is referred to as β-reduction, it will be covered formally later in the chapter.

1.1 The Language of λ-Calculus

Definition 1.1.1. The set of all lambda terms Λ is defined inductively as follows:

- (Variable) If $x \in \mathcal{V}$, then $x \in \Lambda$.
- (Abstraction) If $x \in \mathcal{V}$ and $M \in \Lambda$, then $(\lambda x.M) \in \Lambda$.
- (Application) If $M, N \in \Lambda$, then $(MN) \in \Lambda$.

Where $\mathcal{V} = \{x, y, z, ...\}$ represents a countably infinite set of variable names. These notions are conveniently synthesized into the following abstract syntax:

$$\Lambda := \mathcal{V} \mid \lambda x.\Lambda \mid \Lambda\,\Lambda$$

that directly translates into the following Haskell snippet:

[1] Let p be a sufficiently large prime, and let $f, g : \mathbb{Z}_p \to \mathbb{Z}_p$ be defined by $f(x) = x^2$ and $g(x) = \log_a(a^{x+2})$, where $a \in \mathbb{Z}_p^\times$ is a fixed primitive root. Although f and g are extensionally equal, i.e., they yield the same output for all inputs in $\mathbb{Z}_p$, they are intensionally distinct. The function f performs a simple squaring operation, while g requires evaluating a discrete logarithm, intractable in general Nadal 2021.

```
data Term = Var String
      | Abs String Term
      | App Term Term
```

The key takeaway of this definition is that abstraction and application when paired with a reduction procedure, capture the meaning of function in a way that is abstract yet useful, and enables computation.

Notation 1. We use uppercase letters, e.g. M, N, L, P, Q to denote elements of Λ and lowercase e.g. x, y, z to denote elements of $\mathcal{V}$.

Remark 1. From the definition above we extract that $\lambda x.x{+}1 \notin \Lambda$, given this circumstance, the need to find an accurate representation for numbers within this definition arises. See Definition 1.4.3

Example 1.1.1. Some examples of valid λ terms generated using the syntax from 1.1.1: y, $(\lambda x.(xx))$, $(\lambda x.(\lambda y.x))$, $(((\lambda x.(xy))(\lambda y.y))z)$. The Haskell counterpart for the second term is:

```
Abs "x" (Abs "y" (Var "x"))
```

As a convention to avoid notational cluttering, the outermost parenthesis can be omitted e.g. $\lambda x.x$ is read as $(\lambda x.x)$. Also, application is left-associative and binds tighter than abstraction, so MNP is parsed as $(MN)P$ and $\lambda x.MN$ means $\lambda x.(MN)$, not $(\lambda x.M)N$. Likewise, abstraction is right-associative e.g. $\lambda x.\lambda y.M$ is $\lambda x.(\lambda y.M)$ and binds more weakly than application. Haskell Curry introduced what we now refer to as currying, its main purpose is to avoid unnecessary lambdas. Below is a summary of common notational conventions used throughout the text:

$$\lambda x.x \iff (\lambda x.x) \qquad \lambda x.MN \iff \lambda x.(MN) \qquad MNP \iff (MN)P \qquad \lambda x.\lambda y.f(x,y) \iff \lambda xy.f(x,y)$$

All these are just to keep notation straight-forward, since the formal definition does not leave precedence under-specified thanks to parenthesis. Those interested in the implementation need not worry, as Haskell enforces an explicit order on constructors.

1.2 Reduction of terms

Now that the λ-calculus language has been introduced, it is time we move on to the mechanics of computation, manifested through β-equivalence. α and η equivalence will also be introduced. These two are more difficult to grasp. Informally, it could be said that α-equivalence is equivalence programs up to renaming of local variables. In this same scenario, η-equivalence would singnify equivalence of programs up to wrapping or unwrapping them with an identity function.

Subterms

From the ground up, we define what a subterm is, since the definition of the language is self-referential and terms might contain other terms.

Definition 1.2.1. Given $T \in \Lambda$, $\mathbf{Sub} : \Lambda \to \mathscr{P}(\mathcal{V})$ maps a term to the set of it's subterms.

$$\begin{aligned}
&\mathbf{Sub}(x) = \{x\} \\
&\mathbf{Sub}(MN) = \mathbf{Sub}(M) \cup \mathbf{Sub}(N) \cup \{MN\} \\
&\mathbf{Sub}(\lambda x.M) = \mathbf{Sub}(M) \cup \{\lambda x.M\}
\end{aligned}$$

Where $x \in \mathcal{V}$, $M, N \in \Lambda$ and $\mathscr{P}$ denotes the power set. In Haskell:

```
subTerms :: Term -> Set.Set Term
subTerms (Var x) = Set.singleton (Var x)
subTerms (App m n) =
    Set.insert (App m n) (Set.union (subTerms m) (subTerms n))
subTerms (Abs x m) =
    Set.insert (Abs x m) (subTerms m)
```

Remark 2. Let $\preceq$ be the subterm relation on λ-terms, defined by $M \preceq N \iff M \in \mathbf{Sub}(N)$. $\preceq$ is a partial order relation. This statement is follows intuitively from the definition, it can be proven using induction on the derivation tree.

Definition 1.2.2. A term $S \in \mathbf{Sub}(M)$, $M \in \Lambda$ is said to be proper if $S \neq M$. $\mathbf{Sub}(M) \setminus \{M\}$ would be the set of proper subterms.

Definition 1.2.3. Let $M \equiv \lambda x.N$ be a λ-term:

- The variable x is the binding variable of the abstraction $\lambda x.N$.
- An occurrence of a variable x in M is bound if $x \in \mathbf{Sub}(\lambda x.N)$.
- Free variables are those that are not bound.

Example 1.2.1. Take $M \equiv \lambda x.\, x\, y$, in this example, the x in $\lambda x.$ is binding, y is free, and x is bound. This means that y is used literally, and that x, as denoted by $\lambda x.$ is abstracted and thus subject to being replaced within the context of said abstraction. A subexample might make this explicit: $(\lambda x.\, x\, y)\, M \rightsquigarrow_\beta M\, y$. The y remained while the x was used as a placeholder for M.

Note 1. This taxonomy is key to understanding concepts such as α-equivalence intuitively.

Free Variables

The set of Free Variables **FV**, those that will not get replaced during the process of reduction, is computed using:

Definition 1.2.4. Given $T \in \Lambda$, $\mathbf{FV} : \Lambda \to \mathscr{P}(\mathscr{V})$ outputs the set of free variables for T:

$$\begin{aligned}
\mathbf{FV}(x) &= \{x\} \\
\mathbf{FV}(MN) &= \mathbf{FV}(M) \cup \mathbf{FV}(N) \\
\mathbf{FV}(\lambda x.M) &= \mathbf{FV}(M) \setminus \{x\}
\end{aligned}$$

Where $x \in \mathscr{V}$ and $M, N \in \Lambda$. In Haskell:

```
freeVars :: Term -> Set.Set String
freeVars (Var x) = Set.singleton x
freeVars (App m n) = Set.union (freeVars m) (freeVars m)
freeVars (Abs x m) = Set.delete x (freeVars m)
```

Example 1.2.2. Computing the free variables of $\lambda x.\lambda y.yxz$. The term well-formed, so we proceed:

$$\begin{aligned}
\mathbf{FV}(\lambda x.\lambda y.yxz) &= \mathbf{FV}(\lambda y.yxz) \setminus \{x\} \\
&= \mathbf{FV}(yxz) \setminus \{x, y\} \\
&= \{x, y, z\} \setminus \{x, y\} \\
&= \{z\}
\end{aligned}$$

Of course, z is the only free variable in the expression, since it is the only variable that is not captured by some λ-abstraction.

Definition 1.2.5. A term $M \in \Lambda$ is closed if and only if $\mathbf{FV}(M) = \emptyset$, closed lambda terms are often referred to as combinators. Λ^0 is the set of all closed lambda terms.

Note 2. They are called combinators since having no free variables means that all variables are bound, and thus, a closed term just combines bound variables.

Variable Substitution

We approach the definition for α-equivalence to this end we have to introduce what it means to substitute a variable.

Definition 1.2.6. Capture-Avoiding Substitution Rules Let $M, N \in \Lambda$ and $x, y, z \in \mathscr{V}$. The substitution $M[x := N]$ is defined inductively as:

$$\begin{aligned}
x[x := N] &= N \\
y[x := N] &= y && y \neq x \\
(M_1\, M_2)[x := N] &= (M_1[x := N])\,(M_2[x := N]) \\
(\lambda x.M)[x := N] &= \lambda x.M \\
(\lambda y.M)[x := N] &= \lambda y.(M[x := N]) && y \neq x, y \notin \mathbf{FV}(N) \\
(\lambda y.M)[x := N] &= \lambda z.(M[y := z][x := N]) && y \neq x, y \in \mathbf{FV}(N)
\end{aligned}$$

where $z \notin \mathbf{FV}(M) \cup \mathbf{FV}(N)$. Implementing the latter in Haskell requires a bit more code than previous definitions, along with making some practical tradeoffs:

```
subst :: String -> Term -> Term -> Term
subst x s v@(Var y)
    | x == y = s
    | otherwise = v
subst x s l@(Abs y body)
    | x == y = l
    | Set.notMember y (freeVars s) = Abs y (subst x s body)
    | otherwise =
        let used = Set.unions [allVars body, allVars s, Set.singleton x]
            z = freshVar used
            body' = rename y z body
        in Abs z (subst x s body')
subst x s (App a b) = App (subst x s a) (subst x s b)
```

Note 3. `freshVar` generates a new name for binding variables such that $z \notin \mathbf{FV}(M) \cup \mathbf{FV}(N)$. More details are available in the source code.

Hopefully the meaning of the first three rules is clear, they ancapsulate renaming in the context of variables and applications. The subsequent rules are about abstractions, rule four states that renaming a binding variable has no effect, renaming only modifies a term whenever the renamed variable is different from the binding variable. Rule five is the general case, that is, when the substitution just moves in together with the abstracted term. Then there is the last case, when the binding variable is a bound occurrence in the term we are substituting, which poses a problem, to see why, let us work through an example:

Example 1.2.3. If we mistakenly substitute x by $(\lambda w.\, w\, y)$ in $(\lambda y.\, y\, x)$ using rule five, we get:

$$(\lambda y.\, yx)[x := (\lambda w.\, w\, y)] = (\lambda y.\, y\, (\lambda w.\, w\, y))$$

This is problematic since the free variable y became bound after the rename. Free and bound variables should maintain their freedom or boundedness after a rename. The correct way to substitute avoiding the name clash:

$$(\lambda y.\, yx)[x := (\lambda w.\, w\, y)] = (\lambda z.\, z\, (\lambda w.\, w\, y))$$

Binding variable serve as placeholders, therefore, renaming should not affect the integrity of an expression.

Note 4. Hence the use of Capture Avoiding alongside Substitution.

One notable result regarding substitution is the substitution lemma:

Lemma 1.2.1. Let M, N, P be lambda terms and x, y be variables. Then:

$$M[x := N][y := P] = \begin{cases} M[x := N[y := P]], & \text{if } x = y, \\ M[y := P][x := N[y := P]], & \text{if } x \neq y \text{ and } y \notin \mathrm{FV}(N). \end{cases}$$

Proof Sketch. By structural induction on M, using α-conversion to avoid capture and assuming bound variables x, y, N, P are fresh.

- Base case: Direct from the definition of substitution; check $z = x$, $z = y$, and z distinct from both.
- Inductive step on applications, where substitution distributes: $(M_1 M_2)[x := N] = M_1[x := N]\, M_2[x := N]$, so both statements follow immediately from the induction hypothesis on M_1 and M_2.
- Inductive step on abstractions with z fresh, $(\lambda z.M_0)[x := N] = \lambda z.(M_0[x := N])$, and the induction hypothesis applies to M_0.

□

In the previous example a bound variable is renamed to avoid a naming collision and avoid binding a previously free variable, α-equivalence is the generalization of this technique, it defines equivalence up to renaming of binding variables.

Notation 2. In many of the upcoming definitions inference rules are used, here is a simple introduction for those unfamiliar with the concept:

$$\frac{P_1 \quad P_2 \quad \cdots \quad P_n}{C}\ \textsc{Name}$$

where P_i are judgments assumed to hold, C is the judgment inferred from the premises and NAME is an optional identification label.

α-equivalence

Definition 1.2.7. Two lambda terms $M, N \in \Lambda$ are α-equivalent, written $M =_\alpha N$, if they are structurally identical except for the names of bound variables. Formally:

$$\frac{}{\lambda x.\, L =_\alpha \lambda y.\, L[x := y]}\ \textsc{Alpha} \qquad \frac{L =_\alpha L'}{\lambda x.\, L =_\alpha \lambda x.\, L'}\ \textsc{Abs}$$

$$\frac{L_1 =_\alpha L_1'}{L_1\, L_2 =_\alpha L_1'\, L_2}\ \textsc{AppL} \qquad \frac{L_2 =_\alpha L_2'}{L_1\, L_2 =_\alpha L_1\, L_2'}\ \textsc{AppR}$$

α-equivalence is an equivalence relation:

(Reflexivity)	$M =_\alpha M$
(Transitivity)	$M =_\alpha N$ and $N =_\alpha P \Rightarrow M =_\alpha P$
(Symetry)	$M =_\alpha N \Rightarrow N =_\alpha M$

Example 1.2.4. A simple equivalence class and two samples of not α-equivalent pairs.

$$\lambda x.\lambda y.\, y =_\alpha \lambda a.\lambda b.\, b =_\alpha \lambda z.\lambda x.\, x$$

$$\lambda x.\lambda y.\, x \neq_\alpha \lambda x.\lambda y.\, y \quad \lambda x.\, x\, y \neq_\alpha \lambda x.\, x\, z$$

Remember that terms are α-equivalent only when bounded names are properly substituted.

Remark 3. α-equivalence comes naturally once substitution is defined, since the names of bound variables are just placeholders that will eventually hold an argument.

Note 5. Variable naming is a nightmare. De Bruijn indexing is a way of methodically assigning numbers to variable names, indicating how many binders away a variables binder is. This system eliminates the need for α equivalence to exist.

$$\lambda x.\, x \Rightarrow \lambda.0 \quad \lambda x.\lambda y.\, x \Rightarrow \lambda.\lambda.1 \quad \lambda x.\lambda y.\, y \Rightarrow \lambda.\lambda.0$$

We are better at names than we are at numbers, which is why we will prefer using symbols over De Brujn indexing. Programs like automated theorem provers and compilers do use similar naming strategies to simplify their work.

β-equivalence

Defines the means to perform computation in the λ-calculus, applying substitution as a means of reduction for expressions. This is similar to how one simplifies an arithmetic expression, only it is more powerful, since arithmetic expressions lack expressiveness. It fails to reproduce branching for instance.

Definition 1.2.8. Single-step β-reduction $\rightsquigarrow_\beta$ is defined using the following inference rules:

$$\frac{}{(\lambda x.\, M)\, N \rightsquigarrow_\beta M[x := N]}\ \textsc{Beta} \qquad \frac{M \rightsquigarrow_\beta M'}{\lambda x.\, M \rightsquigarrow_\beta \lambda x.\, M'}\ \textsc{Abs}$$

$$\frac{M_1 \rightsquigarrow_\beta M_1'}{M_1\, M_2 \rightsquigarrow_\beta M_1'\, M_2}\ \textsc{AppL} \qquad \frac{M_2 \rightsquigarrow_\beta M_2'}{M_1\, M_2 \rightsquigarrow_\beta M_1\, M_2'}\ \textsc{AppR}$$

where $M, N \in \Lambda$. In Haskell:

```
step :: Term -> Maybe Term
step (App (Abs x m) n) = Just (subst x n m)
step (App a b) = case step a of
    Just a' -> Just (App a' b)
    Nothing -> case step b of
        Just b' -> Just (App a b')
        Nothing -> Nothing
step (Abs x t) = case step t of
    Just t' -> Just (Abs x t')
    Nothing -> Nothing
step (Var _) = Nothing
```

Single β-reductions can be applied succesively, which induces the following definition

Definition 1.2.9. Multi-step β-reduction. We write $M \rightsquigarrow_\beta^* N$ if and only if there exists a sequence of one-step β-reductions starting from M and ending at N.[2]

$$M \equiv M_0 \rightsquigarrow_\beta M_1 \rightsquigarrow_\beta M_2 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta M_{n-1} \rightsquigarrow_\beta M_n \equiv N$$

That is, there exists an integer $n \geq 0$ and a sequence of terms $M_0, M_1, \ldots, M_n$ such that: $M_0 \equiv M$ and $M_n \equiv N$ for all i with $0 \leq i < n$, we have $M_i \rightsquigarrow_\beta M_{i+1}$.

The process of succesively β-reducing a term leads raises the question of whether all terms can be reduced indefinitly. We know the answer to this, since for instance, variables in $\mathcal{V}$ cannot be reduced. The subsequent question is then, Do all all terms have some kind of normal form that cannot be reduced further? This is answered in Section 1.5. We now introduce a formal definition of normal form that will later allow us to gain insight into the normalization properties of the λ-calculus.

Definition 1.2.10. A λ-term M is in normal form if there is no term N such that $M \rightsquigarrow_\beta N$. Equivalently, M contains no subterm of the form $(\lambda x.P)\,Q$ that can be reduced by beta-reduction. We say a term M is irreducible under beta-reduction.

Definition 1.2.11. A reducible expression, or redex for short, is any subterm of the form $(\lambda x.P)\,Q$. Such a term can be reduced by a single beta-reduction step, resulting in $P[x := Q]$.

Definition 1.2.12. β-equivalence relation, equivalence of λ-terms through β-reductions.

$$M =_\beta N \iff M \rightsquigarrow_\beta^* N \vee N \rightsquigarrow_\beta^* M$$

Note 6. $=_\beta$ is the symetric closure of $\rightsquigarrow_\beta^*$.

We now explore a key aspect of the λ-calculus, in the jargon referred to as first-class citizenship, or high-order, a property of terms that restrains them from being exclusively classified as either function or argument. The example below sheds some light on the matter:

Example 1.2.5. High Order by example. Let us β-reduce the λ-term $A\,B\,C$ and inspect the consequences:

$$\begin{aligned}
\overbrace{(\lambda f.\lambda y.f\,f\,y)}^{A}\overbrace{(\lambda x.x+1)}^{B}\overbrace{(2)}^{C} &\rightsquigarrow_\beta (\lambda y.(\lambda x.x+1)(\lambda x.x+1)\,y)(2) \\
&\rightsquigarrow_\beta (\lambda x.x+1)(\lambda x.x+1)(2) \\
&\rightsquigarrow_\beta^* (\lambda x.x+1)(3) \\
&\rightsquigarrow_\beta^* (4)
\end{aligned}$$

Taking a closer look, A, applies f twice to x, as a consequence, B is being applied twice to C. And so, very easily, we have implemented the $x+2$ function via a sequential application of $x+1$ to C.

Remark 4. As seen above, we can pass what we understand as conventional functions as arguments since they are treated alike. One can intuitively apreciate the computational expresivenes this brings with it, and how this syntactic-semantic homogeneity sets the λ-calculus apart from the classical set-theoretic approach to functions.

η-equivalence

This last notion of equivalence, refers to a program being equivalent to itself wrapped with an identity function:

Definition 1.2.13. Two lambda terms M and N are said to be η-equivalent, written $M =_\eta N$, if:

$$\frac{x \notin \mathbf{FV}(M)}{(\lambda x.M\,x) =_\eta M}\ \textsc{Eta} \qquad \frac{M =_\eta M'}{\lambda x.M =_\eta \lambda x.M'}\ \textsc{Abs}$$

$$\frac{M_1 =_\eta M_1'}{M_1\,M_2 =_\eta M_1'\,M_2}\ \textsc{AppL} \qquad \frac{M_2 =_\eta M_2'}{M_1\,M_2 =_\eta M_1\,M_2'}\ \textsc{AppR}$$

[2]The convention in the literature is to use $\rightarrow_\beta$ for β-reductions $\twoheadrightarrow_\beta$ for multi-step β-reductions. We prefer to use $\rightsquigarrow_\beta$ and $\rightsquigarrow_\beta^n$ since this allows us to numerate β-reduction steps e.g. $(\lambda f.\lambda x.x)\,M\,N \rightsquigarrow_\beta^2$ N. Instead of $\rightsquigarrow_\beta^n$, we use $\rightsquigarrow_\beta^*$ whenever the number of steps is undefined.

The abstraction $\lambda x.(M\ x)$ performs the same operation as M, so if x is not used freely within M, the abstraction serves no purpose. This is shown in ETA, the rest of the inference rules propagate this equivalence throughout the whole term.

Note 7. Two terms are observationally equivalent if they cannot be distinguished by any context that can be applied to them. η-equivalence can be understood as a form of weak extensionality[3], since it does not fully encode observational equivalence.

$\beta\eta$-reduction

To later prove the Church–Rosser property, we introduce a set of rules that captures reduction taking extensionality into account, we combine both β and η-reduction into a single relation.

Definition 1.2.14. We write $M \rightsquigarrow_{\beta\eta} N$ if M reduces to N by one application of either a β or an η-rule, possibly in a subterm. The rules are:

$$\frac{}{(\lambda x.M)\,N \rightsquigarrow_{\beta\eta} M[x := N]}\ \textsc{Beta} \qquad \frac{x \notin \mathbf{FV}(M)}{\lambda x.(M\,x) \rightsquigarrow_{\beta\eta} M}\ \textsc{Eta}$$

$$\frac{M_1 \rightsquigarrow_{\beta\eta} M_1'}{M_1\,M_2 \rightsquigarrow_{\beta\eta} M_1'\,M_2}\ \textsc{AppL} \qquad \frac{M_2 \rightsquigarrow_{\beta\eta} M_2'}{M_1\,M_2 \rightsquigarrow_{\beta\eta} M_1\,M_2'}\ \textsc{AppR}$$

$$\frac{M \rightsquigarrow_{\beta\eta} M'}{\lambda x.M \rightsquigarrow_{\beta\eta} \lambda x.M'}\ \textsc{Abs}$$

Definition 1.2.15. We write $M \rightsquigarrow^*_{\beta\eta} N$ if M reduces to N by zero or more single-step $\beta\eta$-reductions, the same way we did for β-reduction in 1.2.9.

1.3 Normalization & The Church-Rosser Theorem

In this section we attempt to give the reader an overview of the properties behind this calculus. We analyze what makes the untyped λ-calculus a suitable computation model, and provide enough evidence that it is missing some desirable properties that motivate the introduction of typing.

The Church-Rosser in the λ-Calculus

To this end we specify some tools general to relations, specifically, those that help us reason about confluence and termination.

Definition 1.3.1. Let $R \subseteq A \times A$ be a binary relation on a set A. The reflexive-transitive closure of R, denoted by R^*, is the smallest relation on A such that:

(Reflexivity)	$(x, x) \in R^*$
(Transitivity)	$(x, y) \in R^* \wedge (y, z) \in R^* \Rightarrow (x, z) \in R^*$
(Inclusion of R)	$(x, y) \in R \Rightarrow (x, y) \in R^*$

where $x, y, z \in A$.

Remark 5. R^* contains all pairs that relate to each other in such a way that x can be transformed into y by applying R zero or more times. Essentially, reflexive transitive closures are chains of relations that relate terms from the first to the last of the chain if there is one.

Example 1.3.1. $\rightsquigarrow^*_{\beta}$ defined in 1.2.9 is the reflexive transitive closure of $\rightsquigarrow_{\beta}$. Equivalently, $\rightsquigarrow^*_{\beta\eta}$ is too the reflexive transitive closure of $\rightsquigarrow_{\beta\eta}$.

[3] Understood as we presented it in the introduction

Definition 1.3.2. Let $\to$ be a binary relation on a set A. The symbol $\to^*$ denotes reflexive-transitive closure of $\to$. We define the Church–Rosser poperty:

$$x \to^* y \wedge x \to^* z \Rightarrow \exists w \in A,\ y \to^* w \wedge z \to^* w$$

where $x, y, z, w \in A$. In a diagram:

$$\begin{tikzcd}
 & x \arrow[dl, "*"'] \arrow[dr, "*"] & \\
y \arrow[dr, dashed, "*"'] & & z \arrow[dl, dashed, "*"] \\
 & w &
\end{tikzcd}$$

Definition 1.3.3. Let $\to$ be a binary relation on a set A. Its reflexive–transitive closure is denoted by $\to^*$. We say that $\to$ has the diamond property if

$$x \to y \ \wedge\ x \to z \ \Rightarrow\ \exists w \in A,\ y \to w \ \wedge\ z \to w,$$

for all $x, y, z, w \in A$.

The following section covers the proof of this property for the untyped λ-calculus. We deemed it necessary to include the proof since it provides evidence that the λ-calculusis a viable system to perform computation with, since it ensures that no matter what reduction path we choose first the reduction can always converge into a unique term. See 1.3.7. This notion of confluence is strengthened when we introduce typing 2.4.

Remark 6. If you have no time to spare, Nagele, Oostrom, and Sternagel 2016 is a mechanized proof of the Church-Rosser Theorem using the Z-property written in Nominal Isabelle, which enables you to swiftly glance over the following section.

Proof Sketch of the Church-Rosser for $\beta\eta$-Reduction

In this section, we prove the Church–Rosser Theorem for $\beta\eta$-reduction as done in Selinger 2008, even though proof is due to Tait and Martin-Löf. We begin by defining a new relation $M \rightrightarrows M'$ on terms, called parallel one-step reduction. It is the smallest relation satisfying the following rules:

$$\frac{P \rightrightarrows P' \qquad N \rightrightarrows N'}{PN \rightrightarrows P'N'}\ \textsc{ParApp} \qquad \frac{N \rightrightarrows N'}{\lambda x.N \rightrightarrows \lambda x.N'}\ \textsc{ParAbs} \qquad \frac{Q \rightrightarrows Q' \qquad N \rightrightarrows N'}{(\lambda x.Q)N \rightrightarrows Q'[x := N']}\ \textsc{ParBeta}$$

$$\frac{P \rightrightarrows P' \qquad x \notin \mathbf{FV}(P)}{\lambda x.(P\,x) \rightrightarrows P'}\ \textsc{ParEta}$$

The relation $\rightrightarrows$ is also reflexive, $x \rightrightarrows x$ for every x.

Remark 7. In this section, we will not provide explicit proofs of the lemmas. Most are straightforward, follow from induction on terms, and their statements are sensible enough to be understood without detailed derivations.

Lemma 1.3.1. For all M, M', if $M \rightsquigarrow_{\beta\eta} M'$ then $M \rightrightarrows M'$.

Lemma 1.3.2. For all M, M', if $M \rightrightarrows M'$ then $M \rightsquigarrow^*_{\beta\eta} M'$.

Lemma 1.3.3. $\rightsquigarrow^*_{\beta\eta}$ is the reflexive, transitive closure of $\rightrightarrows$.

Proof. Section 4.4 from Selinger 2008 proves 1.3.1, 1.3.2, 1.3.3 in full detail. □

The next goal now is to prove that $\rightrightarrows$ satisfies the diamond property, since together with lemma 1.3.3, implies the Church–Rosser Property. To this end, we first define the maximal parallel one-step reduct M^* of a term M as follows:

- $x^* = x$, for a variable.
- $(P\,N)^* = P^*\,N^*$, if $P\,N$ is not a β-redex.

- $((\lambda x.Q)\,N)^* = Q^*[N^*/x]$.
- $(\lambda x.N)^* = \lambda x.N^*$, if $\lambda x.N$ is not an η-redex.
- $(\lambda x.(P\,x))^* = P^*$, if $x \notin \mathbf{FV}(P)$.

Note that M^* depends only on M.

Lemma 1.3.4. Whenever $M \rightrightarrows M'$, then $M' \rightrightarrows M^*$.

Proof. Section 4.4 from Selinger 2008. □

Lemma 1.3.5. If $M \rightrightarrows N$ and $M \rightrightarrows P$, then there exists Z such that $N \rightrightarrows Z$ and $P \rightrightarrows Z$.

Proof. Take $Z = M^*$. □

Theorem 1.3.6. $\rightsquigarrow_{\beta\eta}$ is Church–Rosser.

Proof. Since $\rightrightarrows$ satisfies the diamond property (Lemma 1.3.5), its reflexive–transitive closure $\rightrightarrows^*$ also satisfies the diamond property. By Lemma 1.3.3, $\rightrightarrows^*$ coincides with $\rightsquigarrow^*_{\beta\eta}$. Hence $\rightsquigarrow^*_{\beta\eta}$ satisfies the diamond property, which is exactly the Church–Rosser property for $\rightsquigarrow_{\beta\eta}$. □

Corollary 1.3.7. If a λ-term M reduces to two normal forms N_1 and N_2, then they are unique up to alpha-equivalece:

$$M \rightsquigarrow^*_\beta N_1 \text{ and } M \rightsquigarrow^*_\beta N_2 \text{ and } N_1, N_2 \text{ are normal } \Rightarrow N_1 =_\alpha N_2$$

Proof. By the Church-Rosser theorem, there exists a term P such that $N_1 \rightsquigarrow^*_\beta P$ and $N_2 \rightsquigarrow^*_\beta P$. Since N_1 and N_2 are normal forms, they cannot be reduced further. Thus, the only possible P is $N_1 =_\alpha N_2$. □

Normalization

Normalization properties essentially summarize the termination properties of terms in the context of β-reduction. So, while The Church-Rosser property ensures confluence, normalization focuses on termination i.e. whether reduction eventually terminates.

Definition 1.3.4. Consider a lambda term M and all possible β-reduction sequences

$$M \equiv M_0 \rightsquigarrow_\beta M_1 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta M_n$$

$$M \equiv N_0 \rightsquigarrow_\beta N_1 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta$$

$$\cdots$$

We classify M as follows:

- M is strongly normalizing if every such reduction sequence terminates in a normal form. Equivalently, there is no infinite reduction sequence: $M \equiv M_0 \rightsquigarrow_\beta M_1 \rightsquigarrow_\beta M_2 \rightsquigarrow_\beta \cdots$
- M is weakly normalizing if there exists at least one finite reduction sequence that terminates in a normal form: $M \equiv M_0 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta M_n$
- M is non-normalizing if no reduction sequence starting from M ever reaches a normal form. In this case, every reduction path is either infinite or loops without producing a normal form.

Note 8. Applying our newly found knowledge on Church–Rosser, we know that out of those different reduction sequences, there always exists some other reduction sequence such that the sequences cross paths.

Proposition 1.3.8. A term may be either, weakly normalizing, strongly normalizing or non-normalizing.

Proof. Examples are:

- $\lambda x.x$ is strongly normalizing since it is already in normal form.
- $\Omega = (\lambda x.xx)(\lambda x.xx)$ reduces to itself forever, hence non-normalizing.
- $(\lambda x.y)\,\Omega \rightsquigarrow_\beta y$, so it has a normal form y, but reducing Ω loops, so it is weakly normalizing.

□

Remark 8. Checking whether a term is terminating is undecidable in general, since it is an instance of the halting problem.

Looking at the big picture, we find the Church-Rosser Property to be a non-trivial result that helps us verify that the λ-calculus is a deterministic model of computation. Meaning that if we take a program and run it, it will always give the same output for a given input. Of course, when we say a program, we mean a λ-term, and when we say run, we mean β-reduce. Perhaps the following section enables the reader to better grasp why we informally speak of lambda terms as programs.

Also, we established the untyped λ-calculus to have pretty weak normalization properties, meaning within the space of possible terms we find everything from strongly-normalizing to non-normalizing terms.

1.4 Encodings & Constructions in the λ-Calculus

This section presents a number of conventional constructions that represent key ideas common to computer science and other fields. We are specifically referring to natural numbers, branching statements, booleans and lists among others, as well as the operations over them. These constructions can be used to prove relevant statements concerning the expressiveness of the λ-calculus.

Note 9. We use our Haskell program to bring some computer science constructions to life, which means that some of the details in this section are implementation dependent. In addition, we use the following type aliases to simplify notation: `v = Var; l = Abs; a = App`.

Natural Numbers

We briefly recall the axiomatic characterization of the natural numbers due to Peano. The Peano axioms provide a minimal foundation for arithmetic, based on a constant 0, a successor function S, and induction.

The connection with the λ-calculus arises from the fact that the same structure can be represented purely functionally. By encoding 0, successor, we recover the essential properties of natural numbers without referring to any external notion of arithmetic. This construction, known as Church encoding, illustrates the expressive power of the λ-calculus and provides the groundwork for representing data and computation purely in terms of functions.

Definition 1.4.1. Peano Axioms, the set of natural numbers $\mathbb{N}$

- **(Zero)** $0 \in \mathbb{N}$
- **(Successor)** $\forall n \in \mathbb{N}, S(n) \in \mathbb{N}$
- (Initial) $\forall n \in \mathbb{N}, S(n) \neq 0$
- (Injectivity) $\forall n, m \in \mathbb{N}, S(n) = S(m) \Rightarrow n = m$
- (Induction) $P(0) \wedge \forall (P(n) \Rightarrow P(S(n))) \Rightarrow \forall n, P(n)$

The first four axioms can be summarized using a graph:

Example 1.4.1. Say for instance we mean to represent 3 using Peano notation, using the graph, we would come up with $s(s(s(0)))$, since it is 3 steps away from 0.

Definition 1.4.2. To add and multiply natural numbers we use following the rules:

$$n + m = \begin{cases} n & \text{if } m = 0 \\ s(n + m') & \text{if } m = s(m') \end{cases} \qquad a \cdot b = \begin{cases} 0 & \text{if } b = 0 \\ a \cdot b' + a & \text{if } b = s(b') \end{cases}$$

Definition 1.4.3. Representation of the set of all naturals in the λ-calculus, usually referred to as Church numerals:

$$\overline{0} \equiv \lambda f.\lambda x.x \qquad \texttt{SUCC} \equiv \lambda n.\lambda f.\lambda x.f(nfx)$$

Using our Haskell program, $\overline{0}$ is represented like follows:

```
zero  = l "f" (l "x" (v "x"))
```

Example 1.4.2. The church numerals have been defined similarly to the peano axioms, that is, by bringing into existance an initial element and using the succesor function to define the rest of them. Let us apply `SUCC` a few times to illustrate:

$$\overline{0} \equiv \lambda f.\lambda x.x \qquad \overline{1} \equiv \lambda f.\lambda x.fx \qquad \overline{2} \equiv \lambda f.\lambda x.f(fx) \qquad \overline{\mathtt{n}} \equiv \lambda f.\lambda x.f^n x$$

where n denotes the number of times we append f.

Notation 3. We use $\overline{\text{number}}$ to denote a number is church encoded. In our Haskell program we use `nat` to convert from Haskell's prelude type `Natural` to type our custom type `Term`.

```
nat :: Natural :: Term
nat n = normalize (nat_ n)
    where
        nat_ 0 = zero
        nat_ n = a succ_ (nat_ (n - 1))
```

Remark 9. Since λ-terms are first-class citizens, and f and x are abstracted away, the reality is that numbers are a quite general construct that has many distinct applications. Recall Example 1.2.5 where $A \equiv_\alpha \overline{2}$.

Basic Arithmetic

Definition 1.4.4. Now that we have a construction for the naturals, we can define some arithmetic operations:

$$\mathtt{ADD} \equiv \lambda m.\lambda n.\lambda f.\lambda x.mf(nfx) \qquad \mathtt{MUL} \equiv \lambda f.\lambda x.\lambda f.\lambda x.m(nf)x$$

$$\mathtt{ISZERO} \equiv \lambda n.n\,(\lambda x.\mathtt{FALSE})\,\mathtt{TRUE} \qquad \mathtt{PRED} \equiv \lambda n.\lambda f.\lambda x.\big(((n\,(\lambda g.\lambda h.h\,(g\,f)))\,(\lambda u.x))\,(\lambda u.u)\big)$$

`PRED` computes the predecessor of a church encoded natural. $\mathtt{PRED}\,\overline{0} = \overline{0}$. We omit further Haskell implementation of constructions from now on since they are just literal translations to our programs representation of λ-terms.

Example 1.4.3. Now we illustrate how addition works by performing $2+3$ with the system we defined.

$$\begin{aligned}
\mathtt{ADD}\;2\;3 &\equiv (\lambda m.\lambda n.\lambda f.\lambda x.m\,f\,(n\,f\,x))\;(\lambda f.\lambda x.f\;f\;x)\;(\lambda f.\lambda x.f\;f\;f\;x)\\
&\rightsquigarrow^*_\beta \lambda f.\lambda x.\;(\lambda f.\lambda x.f\;f\;x)\;f\;((\lambda f.\lambda x.f\;f\;f\;x)\;f\;x)\\
&\rightsquigarrow^*_\beta \lambda f.\lambda x.\;(\lambda x.f\;f\;x)\;((\lambda x.f\;f\;f\;x)\;x)\\
&\rightsquigarrow_\beta (\lambda f.\lambda x.\;f\;f\;f\;f\;f\;x)\\
&\equiv 5
\end{aligned}$$

In Haskell:

```
ghci> normalize (a (a mul (nat 2)) (nat 3))
```

This unequivocally yields `(λf.(λx.f(f(f(f(f(fx)))))))`.

Remark 10. Multiplication works similarly to addition. As for `ISZERO`, the intuition behind it is that if the number is zero, then $\lambda x.\mathtt{FALSE}$ is dropped, and the result is `TRUE`; otherwise, it is applied at least once, which immediately yields `FALSE`.

Remark 11. The λ-calculus is at least capable of arithmetic.

Booleans & Branching

The next constructions in line are the truth values of boolean logic, "*true*" and "*false*" for short.

Definition 1.4.5. Boolean truth values: $\mathtt{TRUE} \equiv \lambda t.\lambda f.t$ and $\mathtt{FALSE} \equiv \lambda t.\lambda f.f$

Remark 12. Note how $\mathtt{TRUE}\;M\;N \rightsquigarrow_\beta M$ and $\mathtt{FALSE}\;M\;N \rightsquigarrow_\beta N$

Definition 1.4.6. Conditional, in other words *"if statement"*. $\mathtt{IF} \equiv \lambda b.\lambda t.\lambda e.b\;t\;e$

Note 10. `IF` statement is just a combinator that happens to behave similarly to a branching statement whenever we pass a church encoded boolean as a first argument.

Example 1.4.4. On the correct behabiour of IF i.e. check that it implements branching. $M, N \in \Lambda$ and IF, TRUE, FALSE are as usual:

$$\begin{aligned} &\texttt{IF TRUE } M\ N \\ &\equiv (\lambda b.\lambda t.\lambda e.\, b\, t\, e)\ \texttt{TRUE}\ M\ N \\ &\rightsquigarrow_\beta (\lambda t.\lambda e.\texttt{TRUE}\ t\, e)\ M\ N \\ &\rightsquigarrow_\beta (\lambda e.\texttt{TRUE}\ M\, e)\ N \\ &\rightsquigarrow_\beta \texttt{TRUE}\ M\, N \rightsquigarrow_\beta^* M \end{aligned} \qquad \begin{aligned} &\texttt{IF FALSE } M\ N \\ &\rightsquigarrow_\beta^* \texttt{FALSE}\ M\, N \rightsquigarrow_\beta^* N \end{aligned}$$

So the IF just combines b, t, e in such a way that whenever b is a church encoded boolean either t or e is dropped depending on the truth value of b. See Remark 12. From this example we extract that $\lambda x.x$ is equivalent to IF it terms of behaviour, and thus, we could use $\texttt{IF} \equiv \lambda x.x$ interchangeably.

Remark 13. Using the IF we can implement an universal set of logic gates:

$$\texttt{NOT} \equiv \texttt{IF}\, b\, \texttt{FALSE}\, \texttt{TRUE} \qquad \texttt{AND} \equiv \texttt{IF}\, a\, (\texttt{IF}\, b\, \texttt{TRUE}\, \texttt{FALSE})\, \texttt{FALSE}$$

This allows us to simulate a nand gate, known to be universal. Thus, the lambda calculus is at least equivalent to propositional logic.

Pairs

A fundamental construct for building data structures is the notion of pairs. They allow us to bind two values together and query them independently.

Definition 1.4.7. $\texttt{PAIR} \equiv \lambda x.\lambda y.\lambda f.f\, x\, y$.

Remark 14. This definition is convenient since $\texttt{PAIR}\, M\, N \rightsquigarrow_\beta^* \lambda f.f\, M\, N$. Now, we can retrieve the first component via $(\lambda f.f\, M\, N)\, \texttt{TRUE}$ and the second via $(\lambda f.f\, M\, N)\, \texttt{FALSE}$. This motivates the following projection operators.

Definition 1.4.8. A PAIR is nothing but an ordered relation $\langle A, B\rangle \in A \times B$. The projections π_1, π_2 are defined as: $\texttt{FST} \equiv \lambda p.p\, (\lambda x.\lambda y.x)$ and $\texttt{SND} \equiv \lambda p.p\, (\lambda x.\lambda y.y)$.

Proposition 1.4.1. A key property of PAIR is: $\texttt{PAIR}\, A\, B \equiv_\beta \texttt{PAIR}\, (\texttt{FST}\, (\texttt{PAIR}\, A\, B))\, (\texttt{SND}\, (\texttt{PAIR}\, A\, B))$.

Lists

Similarly to natural church encoding, lists are treated as a function where: NIL selects the base case, and CONS builds a new list by applying a function to the head and recursing on the tail. This way, operations on lists can be defined using functions alone.

Definition 1.4.9. We define the empty list in the same spirit as 0 and FALSE: $\texttt{NIL} \equiv \lambda f.\lambda z.z$ It represents the terminating element of the inductive definition of lists. To build non-empty lists we use: $\texttt{CONS} \equiv \lambda h.\lambda t.\lambda f.\lambda z.f\, h\, (t\, f\, z)$ which binds a new element h to a pre-existing list t or to NIL.

Definition 1.4.10. Some other relevant list operators are:

$$\texttt{ISNIL} \equiv \lambda l.l\, (\lambda h.\lambda t.\texttt{FALSE})\, \texttt{TRUE}$$

$$\texttt{HEAD} \equiv \lambda l.((l\ (\lambda h.\lambda t.h))\ \texttt{NIL})\texttt{TAIL} \equiv \lambda l.\lambda c.\lambda n.l\ (\lambda h.\lambda r.\lambda g.g\ h\ (r\ c))\ (\lambda c.n)\ (\lambda h.\lambda t.t)$$

1.5 Fixed Point Combinators and Recursion

We have seen that not all terms are normalizing. The most straight forward way to test this, is to check whether there exists some term that reduces to itself, effectively creating a loop in the β-reduction chain so to speak. In more formal terms this means there exists some M such that $M \rightsquigarrow_\beta^n M$ with $n > 0$. As already seen, $\Omega \equiv (\lambda x.\, x\, x)(\lambda x.\, x\, x)$ fullfills this requirement. Let us recall what goes on while β-reducing this term:

$$\Omega \equiv (\lambda x.\, x\, x)(\lambda x.\, x\, x) \rightsquigarrow_\beta (\lambda x.\, x\, x)(\lambda x.\, x\, x) \rightsquigarrow_\beta^* (\lambda x.\, x\, x)(\lambda x.\, x\, x) \equiv \Omega$$

This so called Ω-combinator, proves that not all terms have a normal form, and shines a beam of light onto how recursion can be achieved in the λ-calculus.

There is one caveat to this, that classically, recursion is achieved through naming, take for instance this definition for the gdc:

$$\underline{\text{gcd}}(a,b) = \begin{cases} a & b = 0 \\ \underline{\text{gcd}}(b,\, a \bmod b) & b \neq 0 \end{cases}$$

Here, when $b \neq 0$ the name gcd is used to refer to the same definition again. In the λ-calculus such self-referencing definitions are not permited since the language does not allow names.

Fixed Points

To bypass this setback the notion of fixed point is of particular interest.

Definition 1.5.1. Let $f : X \to X$ be a function. An element $x \in X$ is called a fixed point of f if $f(x) = x$.

Example 1.5.1. Some simple functions that illustrate what fixed points are:

Function	*Fixed Point(s)*
$f(x) = c$	$x = c$
$g(x) = x$	$\{x \mid x \in \mathbb{R}\}$
$h(x) = x^2$	$x = 0,\, x = 1$

This harmless idea satisfies an interesting property, namely that $x = f(x) = f(f(x)) = f^n(x)$ effectively creating a chain where every element $f^{n+1}(x) = f(f^n(x))$, attaining self-reference without naming and showcasing the ability of fixed points to induce recursion. Our quest now, is to find a term that simulates this behavior. To do this, let us translate the definition for fixed point into the world of λ-calculus.

Definition 1.5.2. Given a lambda term $L \in \Lambda$, a term $M \in \Lambda$ is called a fixed point of L if $M =_\beta LM$.

No surprises here, the definition is close to a one-to-one mapping of its functional counterpart seen previously. Interestingly enough, it turns out that every lambda term has a fixed point:

Theorem 1.5.1. For every $L \in \Lambda$, there exists $M \in \Lambda$ such that M is a fixed point for M i.e. $M =_\beta LM$. Referred to as the Fixed Point Theorem in the literature.

Proof. Given $L \in \Lambda$, define $M := (\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x))$, since M is reducible:

$$\begin{aligned} M &\equiv (\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x)) \\ &\rightsquigarrow_\beta L((\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x))) \\ &\equiv L\,M. \end{aligned}$$

Hence, $M =_\beta LM$, as required. □

Remark 15. See how LM is the application of M to L and that using APPR we can keep β-reducing M the same way we did for $x = f(x) = f(f(x))\ldots$ only this time using lambda terms:

$$M \rightsquigarrow_\beta L\,M \rightsquigarrow_\beta L\,L\,M \rightsquigarrow_\beta^* L^n M.$$

Recursive Combinators

Now that nameless self-reference has been proven to exist within the the λ-calculus, we take a step forward and introduce a few of the most important fixed point combinators and explore their different natures.

Definition 1.5.3. Y Combinator. This is the term that given some other term returns its fixed point.

$$Y \triangleq \lambda f.\, (\lambda x.\, f\,(x\,x))\,(\lambda x.\, f\,(x\,x))$$

Remark 16. Even though the Y combinator is of theoretical interest since it allows us to find recursion within the λ-calculus, it is of little practical under strict evaluation since it always runs into non-termination e.g.

$$\begin{aligned} Y\,F &\equiv (\lambda f.(\lambda x.f\,(x\,x))(\lambda x.f\,(x\,x)))F \\ &\rightsquigarrow_\beta (\lambda x.F\,(x\,x))(\lambda x.F\,(x\,x)) \\ &\rightsquigarrow_\beta F((\lambda x.F\,(x\,x))(\lambda x.F\,(x\,x))) \\ &\rightsquigarrow_\beta F(F((\lambda x.F\,(x\,x))(\lambda x.F\,(x\,x)))) \\ &\rightsquigarrow_\beta \cdots \end{aligned}$$

Under strict evaluation, the argument F is not evaluated until Y is done unraveling i.e. never.

To work around this issue, several other combinators exist, the Z and the Turing combinators are two of the better known ones:

Definition 1.5.4. $Z \triangleq \lambda f.(\lambda x.f\,(\lambda v.x\,x\,v))\,(\lambda x.f\,(\lambda v.x\,x\,v))$

Example 1.5.2. Unfolding the Z Combinator

$$\begin{aligned} Z\,F &\equiv (\lambda f.(\lambda x.f\,(\lambda v.x\,x\,v))\,(\lambda x.f\,(\lambda v.x\,x\,v)))\,F \\ &\rightsquigarrow_\beta (\lambda x.F\,(\lambda v.x\,x\,v))\,(\lambda x.F\,(\lambda v.x\,x\,v)) \\ &\rightsquigarrow_\beta F\,(\lambda v.(\lambda x.F\,(\lambda v.x\,x\,v))\,(\lambda x.F\,(\lambda v.x\,x\,v))\,v) \\ &\equiv F\,(\lambda v.Z\,F\,v) \end{aligned}$$

Remark 17. The extra λv in the final form of $Z\,F$ is essential to delay evaluation in a strict environment. Without it, the recursive call would be applied immediately, and cause non-termination in a call-by-value setting.

Definition 1.5.5. Turing combinator $\Theta \triangleq (\lambda x.\lambda y.y\,(x\,x\,y))\,(\lambda x.\lambda y.y\,(x\,x\,y))$

We now use the Turing combinator to aid us implement an untyped version of the factorial function.

λ-Calculus Implementation of Factorial

Classically, factorial is defined as $n! = \prod_{k=1}^{n} k$, that has the following recursive interpretation $n! = n \cdot (n-1)!$ with $0! = 1$. This section develops a worked out example of how to implement this in the λ-calculus.

Example 1.5.3. Implementing factorial using the acquired toolkit:

$$\underline{\texttt{fact}} = \lambda n.\,\texttt{if}\,(\texttt{iszero}\,n)\,(\overline{1})\,(\texttt{mult}\,n\,(\underline{\texttt{fact}}\,(\texttt{pred}\,n))),$$

As already seen, the problem with a recursive definition such as this one is that the name `fact`, appears both on the left and the right-hand side. This means finding `fact` requires solving an equation, which we will do thanks to fixed points. We start by abstracting away `fact` from the term:

$$\texttt{fact} = \overbrace{\left(\lambda f.\lambda n.\,\texttt{if}\,(\texttt{iszero}\,n)\,(1)\,(\texttt{mult}\,n\,(f\,(\texttt{pred}\,n)))\right)}^{F}\,\texttt{fact}.$$

Then the last equation becomes $\texttt{fact} = F\,\texttt{fact}$, which is a fixed point equation. We can solve it up to β-equivalence by letting $\texttt{fact} = \Theta F$. Using our Haskell program we define `fact`:

```
fact = a turing
    (l "f" (l "n"
        (a (a (a if_ (a iszero (v "n"))) (nat 1))
        (a (a mul (v "n"))
            (a (v "f")
            (a pred_ (v "n")))))
    ))
```

We run the following command to compute $\texttt{fact}\,\overline{3}$:

```
ghci> normalize (a fact (nat 3))
```

That of course yields $\lambda f.\lambda x.f\,(f\,(f\,(f\,(f\,(f\,x))))) \equiv \overline{6}$ as result.

Note 11. The latter implementation exhibits a striking syntactic resemblance to that of the Lisp family of languages.

Conclusions

We have covered some of the most important notions that surround the untyped λ-calculus. It has been shown to satisfy the Church-Rosser property of confluence, it has also been shown that this property is not the strongest of its kind. We have also seen that it is not strongly normalizing. For this reasons, it is in our best interest to add some spice to the λ-calculus by means typing. This is to try and make it comply with some stronger requirements that will make it more practical to perform computation. We covered untyped λ-calculus the next chapters focuses on minimal typing variations.

- **Untyped**: Where the type of expressions is never specified. As we have seen, this gives us maximal expressiveness at the cost of being unsafe. Informally, this means we are at a risk of composing unrelated terms that produce miscalculations. For instance xM.
- **Minimal Typing**: Every expression is annotated with a type. This would be equivalent to the usual set theoretic approach to computation where functions have well defined domain and codomain.
- **Extended Typing**: Similar to minimal typing but extended with constructions like products and sums.

This alternatives have dramatically different properties from one another. We will cover them in some detail throughout the text.

References and Resources. This chapter is influenced by many of the classic resources on the subject, primarily Nederpelt and Geuvers 2014 for the basic aspects of the λ-calculus. The proof to the Church-Rosser Theorem and the factorial example are fine-tuned versions of what can be found in Selinger 2008, though, for what it's worth, the factorial implementation is original. Similar proofs are found in O. L. Project 2025 and Nagele, Oostrom, and Sternagel 2016. Other text used as general resources are Henk P. Barendregt and Barendsen 2000 and Girard 1989. We intentionally skipped some of the topics in the literature like combinatory algebras that are not that relevant to our focus of study.

Chapter 2

Typing

We covered the untyped λ-calculus, highlighting its strengths and limitations. This section introduces the notion of typing, which restricts its expressiveness while improving the tractability of studying its properties. The idea of typing arises from the usual definition of a function, understood as a mapping between elements of sets. For example, consider a function $f : A \to B$ defined by $a \mapsto f(a)$, where we recognize A as the domain and B as the codomain. Notice how the mapping is meaningful only within this context–otherwise f would lack sense. Hence, the important of restricting what a function can take as an input as well as what it may yield as an output.

In the context of the λ-calculus, types serve as a means of restriction, similar ho how we use logical predicates to define sets. Just as $f : ? \to ?$ is refined to $f : A \to B$ by specifying requirements on the arguments and ensuring properties of the outputs, we use types to impose requirements on the arguments of an abstraction and to ensure properties of the reduction process of that term.

Since the λ-calculus treats all terms as first-class citizens, attempting to impose restrictions directly on arguments is not effective. Instead, we define restrictions on all terms by pairing each term with additional information called a *type*. This tells us about the nature of a given term, which may be of two kinds: a simple type or arrow type $\to$. Introducing a typing system into the λ-calculus splits Λ into two categories: Typable and Non-typable terms. These sets are disjoint. This simple type system classifies expressions such as $x(\lambda y.y)$ or Y as untypable, while $\lambda x.x$ has type $\alpha \to \alpha$. To summarize, non-normalizing terms and terms that lack meaning are not typable.

Out of the many type systems developed for the -calculus, we focus on a small but expressive core. We begin with minimal typing, which corresponds to the simply typed -calculus with only function types. This system captures the essence of typed functional computation: terms are either variables, abstractions, or applications, and their types are built solely from base types and arrows.

We then extend this core with product types and sum types, which allow richer data structures. Product types model tuples or pairs, supporting construction and projection, while sum types model alternatives, supporting case analysis. These extensions move the type system closer to what is found in practical programming languages, while preserving the foundational simplicity of the -calculus.

2.1 Minimal Typing

The untyped version of the λ-calculus relies on abstraction and application. Minimal typing captures this simply by using base types and arrow types.

Definition 2.1.1. The set of minimal simple types $\mathscr{T}$ is defined inductively as follows:

- (Type variable) If $\alpha \in \mathscr{B}$, then $\alpha \in \mathscr{T}$.
- (Arrow type) If $\sigma, \tau \in \mathscr{T}$, then $(\sigma \to \tau) \in \mathscr{T}$.

where $\mathscr{B}$ is a countably infinite set of base types. In abstract syntax:

$$\mathscr{T} ::= \mathscr{B} \mid (\mathscr{T} \to \mathscr{T})$$

Note 12. These minimal types are the most basic types one can apply to the λ-calculus, this is with respect to extended types, that use products and sums to add expressiveness.

Notation 4. In general, greek letters $\alpha, \beta, \gamma, \ldots$ are standard to denote type names.

Notation 5. The convention is that $\to$ is right associative, that is: $\alpha \to \gamma \to \alpha$ is the same as $(\alpha \to (\gamma \to \alpha))$.

Example 2.1.1. Examples of simple types include:

$$\gamma \quad (\beta \to \gamma) \quad ((\gamma \to \alpha) \to (\alpha \to (\beta \to \gamma)))$$

Definition 2.1.2. A context Γ is a finite mapping from variables to simple types, defined inductively by:

$$\Gamma ::= \cdot \mid \Gamma, x : \sigma$$

where

- $\cdot$ is the empty context
- x is a variable
- $\sigma \in \mathscr{T}$ is a simple type
- $\Gamma, x : \sigma$ extends Γ by mapping x to σ
- $\mathrm{dom}(\Gamma) = \{x \mid x : \sigma \in \Gamma\}$
- $\mathrm{rng}(\Gamma) = \{\sigma \mid x : \sigma \in \Gamma\}$

A context is often represented as a finite sequence: $\Gamma = x_1 : \sigma_1, x_2 : \sigma_2, \ldots, x_n : \sigma_n$ with all x_i distinct.

With this in mind we distinguish two alternative methods for typing. Typing à la Church, where types are annotated alongside λ-terms. And Typing a la Curry, where typing judgements are provided externally through some inference algorithm. Church's style of typing adds syntax on top of the already defined λ-calculus. It can be said that while Church's typing introduces rules to generate well-typed terms, Curry's approach infers types from untyped expressions.

- Church-style: $\lambda x : \sigma . x \vdash \lambda x : \sigma . x : \sigma \to \sigma$.
- Curry-style: $\lambda x . x \vdash \lambda x . x : \sigma \to \sigma$.

Note how Curry's style allows us to assign different valid types to terms. For example, $(\alpha \to \beta) \to (\alpha \to \beta)$ could be a valid type assignment for $\lambda x . x$, do not confuse this polymorphism.

Remark 18. The distinction between Church-style and Curry-style typing has practical consequences for implementation. In Church-style, every abstraction carries its type annotation explicitly, so type checking is straightforward. In Curry-style, terms are written without annotations, and type information must be reconstructed by solving constraints generated during typing. This requires the introduction of type variables and unification algorithms in order to infer consistent typings. Hence Curry-style systems motivate the development of type inference mechanisms, while Church-style systems rely on explicit annotations.

Typing à la Church

Church-style typing embeds type annotations directly in terms, making typing syntax-directed and explicit. We define the following rules that tell whether a term is well-typed.

Definition 2.1.3. We define the following typing relation:

$$\frac{x \notin \mathrm{dom}(\Gamma)}{\Gamma, x : \tau \vdash x : \tau}\ \textsc{Var} \qquad \frac{x \notin \mathrm{dom}(\Gamma) \quad \Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x . M : \sigma \to \tau}\ \textsc{Abs}$$

$$\frac{\Gamma \vdash M : \sigma \to \tau \qquad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau}\ \textsc{App}$$

Example 2.1.2. In this example, we verify that $(\lambda f . \lambda x . f\, x)(\lambda y . y)$ is a valid term of type $\alpha \to \alpha$ using Church-style typing.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma, f : \alpha \to \alpha, x : \alpha \vdash f : \alpha \to \alpha}\ \textsc{Var} \quad \dfrac{}{\Gamma, f : \alpha \to \alpha, x : \alpha \vdash x : \alpha}\ \textsc{Var}}{\Gamma, f : \alpha \to \alpha, x : \alpha \vdash f\, x : \alpha}\ \textsc{App}}{\Gamma, f : \alpha \to \alpha \vdash \lambda x . f\, x : \alpha \to \alpha}\ \textsc{Abs}}{\Gamma \vdash \lambda f . \lambda x . f\, x : (\alpha \to \alpha) \to (\alpha \to \alpha)}\ \textsc{Abs} \quad \dfrac{\dfrac{}{\Gamma, y : \alpha \vdash y : \alpha}\ \textsc{Var}}{\Gamma \vdash \lambda y . y : \alpha \to \alpha}\ \textsc{Abs}}{\vdash (\lambda f . \lambda x . f\, x)(\lambda y . y) : \alpha \to \alpha}\ \textsc{App}$$

Church-style annotations allow type inference to be straightforward because every abstraction carries its parameter type explicitly, this comes at the cost of requiring annotations on all binders, making the language more verbose.

Example 2.1.3. The following implementation of inference on annotated types is witness of the previous statements. We first introduce type annotations on abstractions and define types according to **??**:

```
data Type
    = Base Natural
    | Arr Type Type
data TypedTerm
    = Var String
    | Abs String Type TypedTerm
    | App TypedTerm TypedTerm
```

Then we add the notion of context as a map from variable names to types.

```
type Context = [(String, Type)]
```

We define `infer`:

```
infer :: Context -> TypedTerm -> Maybe Type
infer ctx (Var x) =
  lookup x ctx
infer ctx (Abs x t body) =
  case infer ((x, t) : ctx) body of
    Just bodyType -> Just (Arr t bodyType)
    Nothing -> Nothing
infer ctx (App t1 t2) =
  case infer ctx t1 of
    Just (Arr argType resType) ->
      case infer ctx t2 of
        Just t2Type | t2Type == argType -> Just resType
        _ -> Nothing
    _ -> Nothing

ghci> infer [] (Abs "x" (Base 0) (Var "x"))
Just 0->0
```

Our Church style inference is smart enough to provide us with statements such as:

```
ghci> infer [("y", (Base 0))] (App (Abs "x" (Base 0) (Var "x")) (Var "y"))
Just 0
```

provided that the context carries sufficient information and the expressions are well formed and properly annotated.

Definition 2.1.4. If $\Gamma \vdash M : \sigma$, then we say that M has type σ in Γ. A term M is typable if there exist Γ and σ such that $\Gamma \vdash M : \sigma$.

Typing à la Curry

This typing flavor is probably the more elegant version of the two alternatives presented. It provides complete separation between terms and types, meaning there is no need to extend the syntax of λ-calculus to include type annotations in abstractions. Importantly, in the monomorphic case, if a term is indeed typable, there exists an algorithmic procedure to infer a type for it without requiring explicit type annotations. This approach can be further improved in more advanced systems, sometimes referred to as "Algorithm W", which systematically finds types for typable terms in the polymorphic setting. This idea leads to the Hindley-Milner type system, which allows type inference for terms with polymorphic types.

We implement Typing à la Curry following Nazareth and Nipkow 1996. Algorithm 1 presents a faithful synthesis of their monomorphic version of Algorithm W. We provide a similar alternative that is fully interoperable with the rest of our implementation; however, we omit the code to maintain clarity and readability.

Here, **Var** x and **Abs** e are self-explanatory. For **App** e_1 e_2, the algorithm infers types for the function and argument, unifies them using `mgu`, and returns the resulting type along with the composed substitutions. `TVar(n)` generates fresh type variables, `mgu` computes the most general unifier, and $\circ$ denotes substitution composition.

Algorithm 1 Monomorphic Algorithm W

```
function W(expr, env, n)
    match expr with
      Var x:
    if x ∈ dom(env) then
        return Ok(id, env(x))
    else
        return Fail
    end if
      Abs e:
    (s, t, m) ← W(e, env[x ↦ TVar(n)], n + 1)
    return Ok(s, TVar(n) → t, m)
      App e1 e2:
    (s1, t1, m1) ← W(e1, env, n)
    (s2, t2, m2) ← W(e2, s1(env), m1)
    u ← mgu(s2(t1), t2 → TVar(m2))
    return Ok(u ∘ s2 ∘ s1, u(TVar(m2)), m2 + 1)
end function
```

Example 2.1.4. Using inference algorithms like the one earlier we can provide terms with suitable types without need for explicit type annotations: Let σ, τ, ρ be arbitrary types. Then:

- $\vdash \lambda x.x : \sigma \to \sigma$
- $\vdash \lambda x.\lambda y.x : \sigma \to \tau \to \sigma$
- $\vdash \lambda x.\lambda y.\lambda z.x\,z\,(y\,z) : (\sigma \to \tau \to \rho) \to (\sigma \to \tau) \to \sigma \to \rho$

from our program:

```
ghci> infer [] (Abs "x" (Var "x")) 0
Just ([],(a0 -> a0),1)
ghci> infer [] (Abs "x" (Abs "y" (Var "x"))) 0
Just ([],(a0 -> (a1 -> a0)),2)
```

$\lambda x.\lambda y.\lambda z.x\,z\,(y\,z)$ is omitted for readability. Clearly `a0` corresponds to α and `a1` to τ. Note how the typing context is always empty at first. For testing purposes, let us try and find a type for the Y combinator:

```
ghci> infer [] ycomb 0
Nothing
```

Note 13. Typing style is largely a practical issue, a matter of taste even, since ultimately both approaches yield valid typings. The more we annotate, the less we have to infer, and the converse is also true. For the sake of simplicity, we therefore adopt typing à la Church, keeping type information explicit and straightforward.

Properties of Minimal Typing

In 1.3 we analyzed the untyped λ-calculus as a computation model. We also provided evidence that it is missing some desirable properties, for instance, not all terms reached a normal form, as seen in 1.3, some computations can run forever. This makes reasoning about programs more difficult and motivates the study of typed variants. In the following, we provide a few of the most relevant properties of this minimally typed variant, that wile restricting expressiveness, add safety and decidability. Proofs are omitted, refer to Nederpelt and Geuvers 2014 and H. P. Barendregt 1992.

Definition 2.1.5. The one-step β-reduction relation $\rightsquigarrow_\beta$ on terms of $\Lambda^{\to}$ is the smallest relation satisfying:

- (Basis) $(\lambda x : \sigma.\, M)\, N \rightsquigarrow_\beta M[x := N]$
- (Compatibility) If $M \rightsquigarrow_\beta M'$ then $M\,N \rightsquigarrow_\beta M'\,N, N\,M \rightsquigarrow_\beta N\,M', \lambda x : \sigma.\, M \rightsquigarrow_\beta \lambda x : \sigma.\, M'$

and similarly for all subterms.

Theorem 2.1.1. The ChurchRosser property holds for minimal typing.

Lemma 2.1.2. If $\Gamma \vdash L : \rho$ and $L \to_\beta L'$, then

$$\Gamma \vdash L' : \rho.$$

Theorem 2.1.3. Every legal term M in minimally typed λ-calculusis strongly normalising.

Remark 19. Proofs to these statements are common-ground among most of the λ-calculus resources cited. A proof to the normalization theorem is given for the product extension.

2.2 Extended Typing–Products

As discussed, minimal typing is the simplest form of typing. It follows from this statement that it can be extended. Here we introduce products that can be thought of as records or structs:

Definition 2.2.1. Extended types $\mathscr{T}^\times$ is defined inductively as:

$$\mathscr{T}^\times ::= \mathscr{B} \mid \mathscr{T}^\times \to \mathscr{T}^\times \mid \mathscr{T}^\times \times \mathscr{T}^\times \mid 1$$

where $\mathscr{B}$ is the set of base types.

Definition 2.2.2. The set of raw terms $\Lambda^\times$ is defined by:

$$\Lambda^\times ::= x \mid \lambda x : \mathscr{T}.\ \Lambda^\times \mid \Lambda^\times\ \Lambda^\times \mid \langle \Lambda^\times, \Lambda^\times \rangle \mid \pi_1 \Lambda^\times \mid \pi_2 \Lambda^\times \mid *$$

Here:

- $x \in \mathscr{V}$, a countable set of variables
- $\lambda x : \mathscr{T}.\ \Lambda^\times$ is function abstraction with type annotation
- $\langle \Lambda^\times, \Lambda^\times \rangle$ is the term constructor for product values
- π_1 and π_2 are projections on product terms
- $*$ is the unique term of type 1

Definition 2.2.3. Given $t \in \Lambda^\times$, typing judgments are of the form $\Gamma \vdash t : \mathscr{T}$, where Γ is a context $x_1 : T_1, \ldots, x_n : T_n$. The typing rules for $\Lambda^\times$ are:

$$\frac{\Gamma \vdash m : a \to b \qquad \Gamma \vdash n : a}{\Gamma \vdash m\,n : b}\ \textsc{app} \qquad \frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x : A.\ M : A \to B}\ \textsc{Abs}$$

$$\frac{\Gamma \vdash M : A \qquad \Gamma \vdash N : B}{\Gamma \vdash \langle M, N \rangle : A \times B}\ \textsc{Pair} \qquad \frac{\Gamma \vdash M : A \times B}{\Gamma \vdash \pi_1 M : A}\ \textsc{Proj}_1 \qquad \frac{\Gamma \vdash M : A \times B}{\Gamma \vdash \pi_2 M : B}\ \textsc{Proj}_2$$

$$\frac{}{\Gamma \vdash * : 1}\ \textsc{Unit} \qquad \frac{}{\Gamma, x : A \vdash x : A}\ \textsc{Var}$$

Reduction Rules

The following presents the core operational semantics for the extended types with products and unit. We naturally extend β-reduction for products and projections. We the do the same for η reduction. It is important to close this root rules under a context to form a full reduction relation.

β-reduction

The core β-reduction rules for simply typed λ-calculus with products and unit are:

$$(\lambda x : A.\ M)\ N \rightsquigarrow_\beta M[x := N]$$

$$\pi_1 \langle M, N \rangle \rightsquigarrow_\beta M \qquad \pi_2 \langle M, N \rangle \rightsquigarrow_\beta N$$

$$\langle \pi_1 M, \pi_2 M \rangle = M$$

$$\lambda x : U.\ (M\,x) = M \quad (x \notin \mathrm{FV}(M))$$

η-laws

The η-laws capture extensionality principles, identifying terms with their canonical expansions:

$$\lambda x : A.\, M\, x \equiv_\eta M \quad (x \notin FV(M))$$
$$\langle \pi_1 M, \pi_2 M \rangle \equiv_\eta M$$
$$M \equiv_\eta * \quad (M : 1)$$

Closure: Unlike β, η is usually treated as an equational rule rather than a reduction. It is automatically closed under contexts, since definitional equality is taken as a congruence.

$\beta\eta$-conversion

The combined theory, called $\beta\eta$-conversion, is the smallest congruence $\equiv_{\beta\eta}$ satisfying:

$$M \rightsquigarrow_\beta N \;\Rightarrow\; M \equiv_{\beta\eta} N \qquad M \equiv_\eta N \;\Rightarrow\; M \equiv_{\beta\eta} N$$

That is, $\beta\eta$-conversion identifies terms up to both computation and extensionality, closed under all term formers.

2.3 Church Rosser Revisited

One important theorem that fails for $\beta\eta$-reduction in the simply-typed $\lambda^{\to,\times,1}$-calculus is the ChurchRosser property. The culprit is the η_1-rule. For example, let x be a variable of type $A \times 1$, and consider the term

$$M = \langle \pi_1 x, \pi_2 x \rangle.$$

By the $(\eta_\times)$-rule, M reduces to x, whereas by the (η_1)-rule, M reduces to $\langle \pi_1 x, * \rangle$. Both resulting terms are normal forms, so there is no common reduct. Hence, the ChurchRosser property fails:

$$\langle \pi_1 x, \pi_2 x \rangle \xrightarrow{\eta_1} \langle \pi_1 x, * \rangle, \quad \langle \pi_1 x, \pi_2 x \rangle \xrightarrow{\eta_\times} x.$$

There are two natural ways to restore the ChurchRosser property:

1. If we consider only β-reductions and omit all η-rules, confluence holds. Computationally, this is sufficient: β-reductions drive all actual computation, while η-reductions merely simplify terms. Moreover, η-reductions never block β-redexes and always reduce term size. Hence, any β-normal form can be further reduced to a $\beta\eta$-normal form (though not necessarily uniquely).
2. Alternatively, if we remove the type 1 and the constant $*$ from the calculus, the ChurchRosser property holds even for $\beta\eta$-reduction.

We don't cover the proofs, but this note was necessary given the emphasis given to the notions of normalization and confluence throughout the document.

2.4 Normalization Revisited

The same way we added types and products to the λ-calculus, we now update our previous definition of normal form to adequately adjust to our new requirements:

Definition 2.4.1. A term is normal if none of its subterms is of any of the following forms:

$$\pi_1 \langle u, v \rangle \quad \pi_2 \langle u, v \rangle \quad (\lambda x : u.\, v)\, u$$

since otherwise it would be reducible.

Definition 2.4.2. Let $\to$ be a reduction relation on terms.

- A term M is weakly normalizing if there exists a finite reduction sequence starting from M that terminates in a normal form:
- A term M is strongly normalizing if there are no infinite reduction paths from M:

Remark 20. Strong normalization implies weak normalization.

Theorem 2.4.1. Every well-typed term in the simply typed λ-calculus with product types and unit type is strongly normalizing. Formally, if $\Gamma \vdash M : A$, then there is no infinite sequence of reductions starting from M:

$$M \to M_1 \to M_2 \to \dots$$

Proof. An machine checked proof for strong normalization in **Adgda** can be found at Urciuoli, Tasistro, and Szasz 2020. □

Corollary 2.4.2. The simply typed λ-calculusis weakly normalizing.

The proofs are lengthy and well known. They are covered in full detail by Girard 1989 chapters 4 and 6. The strong normalization theorem is difficult to prove. In contrast, the proof of the weak normalization theorem is not as hard and provides measure of complexity based on the number of redexes of a given degree within a term.

Proof Sketch of the Weak Normalization Theorem

To prove the weak normalization theorem, we assign a numerical measure of complexity to terms based on the degrees of their redexes. By carefully analyzing how these degrees behave under substitution and reduction, we construct an inductive argument showing that every term can be reduced, step by step, to a normal form

Definition 2.4.3. The degree $\partial(T)$ of a type T is defined as:

- $\partial(T_i) = 1$ if T_i is atomic
- $\partial(U \times V) = \partial(U \to V) = \max(\partial(U), \partial(V)) + 1$

Definition 2.4.4. The degree $\partial(r)$ of a redex 2.4.1 r is defined by:

- $\partial(\pi_1 \langle u, v \rangle) = \partial(\pi_2 \langle u, v \rangle) = \partial(U \times V)$, where $U \times V$ is the type of $\langle u, v \rangle$
- $\partial((\lambda x.v)\, u) = \partial(U \to V)$, where $U \to V$ is the type of $(\lambda x.v)$

Definition 2.4.5. The degree $d(t)$ of a term t is the supremum of the degrees of the redexes it contains. For any redex r, we have

$$\partial(r) \leq d(r)$$

since $d(r) = \sup\big(\partial(r), \{\partial(r') \mid r' \subset r\}\big)$

Key Lemmas We cover the necessary lemmas to give the reader an intuition of how the proof develops. We omit the proofs since they are already covered in detail by Girard 1989

Lemma 2.4.3. Degree and Substitution: If x is of type U, then

$$d(t[x := u]) \leq \max(d(t), d(u), \partial(U))$$

The effect substitution has on degree does not blow up and has a limited upper bound. Not only that, we also know the upper bound.

Lemma 2.4.4. Degree and Conversion: If $t \to u$, then

$$d(u) \leq d(t)$$

Reduction only decreases degree.

Lemma 2.4.5. Conversion of Maximal Degree: Let r be a redex of maximal degree n in t, and suppose all redexes strictly contained in r have degree less than n. If u is obtained by converting r to c, then u has strictly fewer redexes of degree n. Meaning that the a term of maximal degree reduces to a term of smaller degree.

Putting the lemmas together, we use the notion of measure to finalize the proof sketch:

Definition 2.4.6. The Measure of a term t, is defined as

$$\mu(t) = (n, m)$$

where: $n = d(t)$ and $m =$ number of redexes of degree n.

Theorem 2.4.6. By iteratively applying Lemma 2.4.5 to reduce the measure $\mu(t)$ in lexicographic order, every term t eventually reaches a normal form. Hence, weak normalization holds.

2.5 Extended Typing–Sums

Sum types can be thought of as disjoint unions, they are the dual to product types, see 4.5.5. Whereas a product type $A \times B$ contains both an A and a B, a sum type $A + B$ contains either an A or a B, together with a tag indicating which side it came from. This makes sum types a natural way to represent alternatives or branching data. As done in Selinger 2008 we only present the extension in the effort avoid being repetitive.

Definition 2.5.1. Extended types $\mathscr{T}^+$ is defined inductively as:

$$\mathscr{T}^+ ::= \mathscr{B} \mid \mathscr{T}^+ \to \mathscr{T}^+ \mid \mathscr{T}^+ + \mathscr{T}^+ \mid 0$$

where $\mathscr{B}$ is the set of base types.

Definition 2.5.2. The set of raw terms Λ^+ is defined by:

$$\Lambda^+ ::= x \mid \lambda x : \mathscr{T}.\ \Lambda^+ \mid \Lambda^+\ \Lambda^+ \mid \iota_1 \Lambda^+ \mid \iota_2 \Lambda^+ \mid \mathsf{case}\ \Lambda^+\ \mathsf{of}\ \iota_1 x \Rightarrow \Lambda^+ \mid \iota_2 y \Rightarrow \Lambda^+ \mid \square_A \Lambda^+$$

Here:

- $x \in \mathscr{V}$, a countable set of variables
- $\iota_1 M$ injects a term of type A into the left summand of $A + B$
- $\iota_2 M$ injects a term of type B into the right summand of $A + B$
- $\mathsf{case}\ M\ \mathsf{of}\ \iota_1 x \Rightarrow N \mid \iota_2 y \Rightarrow P$ performs case analysis on a sum
- $\square_A M$ eliminates an impossible term of type 0, producing a term of type A

Definition 2.5.3. Given $t \in \Lambda^+$, typing judgments are of the form $\Gamma \vdash t : \mathscr{T}$, where Γ is a context $x_1 : T_1, \ldots, x_n : T_n$. The typing rules for Λ^+ are:

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \iota_1 M : A + B}\ \textsc{Inl} \qquad \frac{\Gamma \vdash M : B}{\Gamma \vdash \iota_2 M : A + B}\ \textsc{Inr}$$

$$\frac{\Gamma \vdash M : A + B \qquad \Gamma, x : A \vdash N : C \qquad \Gamma, y : B \vdash P : C}{\Gamma \vdash \mathsf{case}\ M\ \mathsf{of}\ \iota_1 x \Rightarrow N \mid \iota_2 y \Rightarrow P : C}\ \textsc{Case} \qquad \frac{\Gamma \vdash M : 0}{\Gamma \vdash \square_A M : A}\ \textsc{Abort}$$

Reduction Rules

The operational semantics for sum types naturally extend β-reduction with case distinction and the empty type.

β-reduction

The core β-reduction rules for simply typed λ-calculus with sums and empty type are:

$$\mathsf{case}\ (\iota_1 M)\ \mathsf{of}\ \iota_1 x \Rightarrow N \mid \iota_2 y \Rightarrow P \ \rightsquigarrow_\beta\ N[x := M]$$

$$\mathsf{case}\ (\iota_2 M)\ \mathsf{of}\ \iota_1 x \Rightarrow N \mid \iota_2 y \Rightarrow P \ \rightsquigarrow_\beta\ P[y := M]$$

$$\square_A M \ \rightsquigarrow_\beta\ \text{(no rule, } M : 0 \text{ cannot reduce)}$$

η-laws

The η-laws for sums capture the universal property of coproducts:

$$\mathsf{case}\ M\ \mathsf{of}\ \iota_1 x \Rightarrow \iota_1 x \mid \iota_2 y \Rightarrow \iota_2 y \ \equiv_\eta\ M$$

$$\square_A M \ \equiv_\eta\ M \quad (M : 0)$$

Closure: As with products, η is treated as an equational principle, closed under all contexts.

$\beta\eta$-conversion

The combined theory, called $\beta\eta$-conversion, is the smallest congruence $\equiv_{\beta\eta}$ satisfying:

$$M \rightsquigarrow_\beta N \;\Rightarrow\; M \equiv_{\beta\eta} N \qquad M \equiv_\eta N \;\Rightarrow\; M \equiv_{\beta\eta} N$$

Thus, $\beta\eta$-conversion identifies terms up to computation and extensionality, closed under sums and the empty type.

Remark 21. The term constructors for sums have the following interpretation:

- ι_1 and ι_2 are the canonical injections, marking whether a value belongs to the left or right summand of $A + B$.
- The case construct corresponds to case analysis on a coproduct, analogous to pattern matching in functional programming.
- The term abort_A expresses the unique morphism $0 \to A$, capturing the fact that from an impossible value one may derive a term of any type.

2.6 Extended Typing–Sums and Products

The systems $\Lambda^\times$ and Λ^+ may be combined into a single calculus $\Lambda^{\times+}$ with corresponding type system $\mathscr{T}^{\times+}$. In this setting the formation rules for types, raw terms, typing rules, and reduction rules are the union of those previously introduced for sums and for products. Thus,

Definition 2.6.1. Extended types with products and sums, denoted $\mathscr{T}^{\times+}$, are defined inductively as:

$$\mathscr{T}^{\times+} ::= \mathscr{B} \mid \mathscr{T}^{\times+} \to \mathscr{T}^{\times+} \mid \mathscr{T}^{\times+} \times \mathscr{T}^{\times+} \mid \mathscr{T}^{\times+} + \mathscr{T}^{\times+} \mid 1 \mid 0$$

where $\mathscr{B}$ is the set of base types.

Definition 2.6.2. The set of raw terms with products and sums, denoted $\Lambda^{\times+}$, is given by

$$\Lambda^{\times+} ::= \Lambda^\times \cup \Lambda^+.$$

Remark 22. By uniting sums and products we obtain the full simply typed λ-calculus with finite types. Every finite type can be expressed from 1, 0, sums, and products. The typing and operational rules are exactly those already given, taken together without modification.

Kinds of problem in Type Theory

These are the usual problems type theorists face. A programmer should be at least familiar with typability, and even more so with Type Checking, the one responsible for the daily headaches of bug finding in typed languages:

- **Typability**: Determine whether a term is legal. Formally, given a term, find a context and type such that $? \vdash \texttt{term} : ?$ holds. If the term is not well-typed, identify where the derivation fails. Type Assignment is a variant where the context is given, so the task is only to find the type: $\Gamma \vdash \texttt{term} : ?$
- **Type Checking**: Given a context, term, and type, check whether $\Gamma \vdash \texttt{term} : \texttt{type}$ holds. The task is purely to verify that the term has the specified type in the given context.
- **Inhabitation**: Given a context and a type, determine whether there exists a term with that type: $\Gamma \vdash ? : \texttt{type}$

Remark 23. In programming languages and theorem provers, these type problems have concrete meanings. Well-typedness asks whether a term or program is valid under the typing rules, ensuring correctness of proofs or programs. Type Checking verifies that a given term has a specified type, allowing compilers or proof assistants to confirm correctness. Term Finding constructs a term of a given type, corresponding to automatic proof generation in theorem provers or program synthesis in programming languages.

Note 14. In our specific case, the implementations given show an increasing level of type inference. The simplest kind is capable of type checking and relies on annotations for every term, the second implements Church–style typing, and the third implements proper inference on monomorphic types given no annotations. The point of this note is to relate type checking and typability through annotations vs. inference, meaning the more type annotations we add, the more typability reduces to checking.

References & Resources Notions that are ubiquitous to Type Theory have been outsourced from either Nederpelt and Geuvers 2014, Selinger 2008 or Sørensen and Urzyczyn 2006. The subsection on normalization is a synthesis of what can be found in Girard 1989. Other texts that might have influenced the development of this section have been cited accordingly. In regard to implementation details, similar implementations to ours have been examined, namely Nazareth and Nipkow 1996, seen before, and Ahmed 2012. The latter covers formal verification of "Algorithm W" and type inference in the **ML** Language.

Chapter 3

Intuitionistic Logic & Curry–Howard

Unlike classical logic, which is often extended through modal operators or second-order quantifiers, intuitionistic logic takes a different path, not because it adds power, but because it restricts classical reasoning. Classical logic freely allows non-constructive arguments, proofs that assert the existence of an object without providing a way to explicitly construct it. Intuitionistic logic, in contrast, is designed to formalize constructive reasoning, where existence is meaningful only if a specific example can be provided.

3.1 Intuitionistic Logic

To motivate the introduction of Intuitionistic Logic, we present a theorem and two different styles of proof T. O. L. Project 2025:

Theorem 3.1.1. There are irrational numbers a and b such that a^b is rational.

Proof. Let $a, b = \sqrt{2}$

- If $\sqrt{2}^{\sqrt{2}}$ is rational, we are done.
- Otherwise, $\sqrt{2}^{\sqrt{2}}$ is irrational and we can consider $a = \sqrt{2}^{\sqrt{2}}$ and $b = \sqrt{2}$. In which case:

$$a^b = \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^2 = 2,$$

  which is rational so a and b must exist.

□

This is a perfectly valid proof. However, it is somewhat unsatisfying, since it proves existence without exhibiting a specific pair. A more constructive proof is as follows

Proof. Take $a = \sqrt{3}$ and $b = \log_3 4$. Then

$$a^b = \left(\sqrt{3}\right)^{\log_3 4} = 3^{\frac{1}{2}\log_3 4} = \left(3^{\log_3 4}\right)^{1/2} = 4^{1/2} = 2,$$

which is rational. □

Syntax

The syntax for intuitionistic logic is the same as that of propositional logic. Where $\vee, \wedge, \rightarrow$ are used as binary connectives to compose logical statements and $\neg$ is used to negate such statements.

Definition 3.1.1. The set of formulas of propositional intuitionistic logic is defined as follows:

- $\perp$ is an atomic formula.

- Every propositional variable p_i is a formula.
- If φ and ψ are formulas, then $(\varphi \wedge \psi)$ is a formula.
- If φ and ψ are formulas, then $(\varphi \vee \psi)$ is a formula.
- If φ and ψ are formulas, then $(\varphi \to \psi)$ is a formula.

In addition to the connectives introduced above, the following symbols define the following symbols to simplify notation:

- $\neg\varphi$ abbreviates $(\varphi \to \bot)$.
- $\varphi \leftrightarrow \psi$ abbreviates $(\varphi \to \psi) \wedge (\psi \to \varphi)$.

The Brouwer–Heyting–Kolmogorov Interpretation

The Brouwer–Heyting–Kolmogorov Interpretation interpretation provides an informal, constructive understanding of intuitionistic connectives. It is based on the notion of a construction, which can be regarded as a constructive proof. The BHK interpretation assigns meanings to the intuitionistic connectives as follows:

- We assume that we know what constitutes a construction for an atomic proposition.
- A construction of $\varphi_1 \wedge \varphi_2$ is a pair $\langle M_1, M_2 \rangle$, where M_1 is a construction of φ_1 and M_2 is a construction of φ_2.
- A construction of $\varphi_1 \vee \varphi_2$ is a pair $\langle s, M \rangle$, where $s = 1$ and M is a construction of φ_1, or $s = 2$ and M is a construction of φ_2.
- A construction of $\varphi \to \psi$ is a function that converts any construction of φ into a construction of ψ.
- There is no construction for $\bot$; it represents absurdity.
- Defined as $\varphi \to \bot$. Thus, a construction of $\neg\varphi$ is a function that converts a construction of φ into a construction of $\bot$.

Remark 24. Generally speaking, $\neg\varphi$ means a construction of φ is impossible.

Natural Deduction

The syntax that to create formulas is not sufficient, we need to pair it with a system that enables the deduction of other formulas from a set of given ones.

Definition 3.1.2. A deductive system is a pair $\mathscr{D} = (\mathscr{A}, \mathscr{R})$, where:

- $\mathscr{A} \subseteq \mathscr{F}$ is a set of axioms and $\mathscr{F}$ is the set of formulas.
- $\mathscr{R}$ is a set of inference rules used to derive new formulas from the already known ones.

A derivation or proof in $\mathscr{D}$ from a set of premises $\Gamma \subseteq \mathscr{F}$ is a finite sequence of formulas $\varphi_1, \ldots, \varphi_n$ such that each φ_i is either:

- an axiom ($\varphi_i \in \mathscr{A}$),
- a member of Γ, or
- obtained from previous formulas in the sequence by an inference rule in $\mathscr{R}$.

We write $\Gamma \vdash \varphi$ if there exists a derivation of φ from Γ in the deductive system $\mathscr{D}$.

Natural deduction is a deduction system within intuitionistic logic that uses constructive reasoning to derive conclusions. This structured proofs emphasize the existence of explicit constructions rather than abstract truth values.

The rules of natural deduction for intuitionistic propositional logic (without $\bot C$) can be presented as follows:

Introduction / Elimination Rules

$$\frac{\phi \qquad \psi}{\phi \wedge \psi}\ \textsc{IntroConj} \qquad \frac{\phi \wedge \psi}{\phi}\ \textsc{ElimConj} \qquad \frac{\phi \wedge \psi}{\psi}\ \textsc{ElimConj}$$

$$\frac{\phi}{\phi \vee \psi}\ \textsc{IntroDisj} \qquad \frac{\psi}{\phi \vee \psi}\ \textsc{IntroDisj} \qquad \frac{\phi \vee \psi \qquad [\phi]^n \cdots \chi \qquad [\psi]^n \cdots \chi}{\chi}\ \textsc{ElimDisj}$$

$$\frac{[\phi] \cdots \psi}{\phi \to \psi}\ \textsc{IntroImpl} \qquad \frac{\phi \qquad \phi \to \psi}{\psi}\ \textsc{ElimImpl}$$

$$\frac{[\phi]^n \cdots \bot}{\neg\phi}\ \textsc{IntroNeg} \qquad \frac{\phi \qquad \neg\phi}{\bot}\ \textsc{ElimNeg}$$

$$\frac{\bot}{\phi}\ \textsc{ElimBot}$$

Example 3.1.1. some deduction example

3.2 The Curry-Howard Isomorphism

The λ-calculus, as introduced in the first section, provides a minimal but expressive framework for modeling computation. Independently, intuitionistic logic offers a formal system for constructive reasoning that requires the existence of witnesses for propositions. The Curry-Howard correspondence reveals a structural equivalence between them, providing a unifying perspective that connects proofs with programs. The Curry–Howard correspondence establishes the following associations:

- **Propositions ↔ Types**: Proposition in intuitionistic logic can be interpreted as a type in a typed λ-calculus.
- **Proofs ↔ Programs**: Constructive proofs of a proposition correspond to a λ-term inhabiting the associated type.
- **Proof normalization ↔ Program evaluation**: The process of simplifying proofs corresponds to the process of evaluating a term through β-reduction.

Given a proposition P in intuitionistic logic, constructing a proof of P is equivalent to constructing a term t such that $t : T$, where T is the type corresponding to P. This yields the principle often summarized as *"proofs-as-programs"* or *"propositions-as-types"*. The specifics of this relation are summarized in table 3.1.

Table 3.1: The Curry–Howard Correspondence

Intuitionistic Logic	**Typed λ-Calculus**
Proposition A	Type A
Proof of A	Term of type A
Implication $A \to B$	Function type $A \to B$
Conjunction $A \wedge B$	Product type $A \times B$
Disjunction $A \vee B$	Sum type $A + B$
$\top$	1
$\bot$	0
Proof normalization	β-reduction

Example 3.2.1. Consider the implication $A \to B$ in intuitionistic logic. A proof of $A \to B$ is a construction that, given a proof of A, produces a proof of B. Under the Curry–Howard correspondence, this directly maps to a function type

$A \to B$ in λ-calculus, where a λ-term $f : A \to B$ takes an argument of type A and returns a result of type B. Similarly, conjunction $A \wedge B$ corresponds to a product type $A \times B$, and disjunction $A \vee B$ corresponds to a sum type $A + B$. In each case, the rules for constructing and deconstructing proofs mirror the operations on the corresponding λ-terms.

Example 3.2.2. Modus Ponens via Curry–Howard. The logical rule for modus ponens is:

$$\frac{P \to Q \quad P}{Q}\ \text{MP}$$

Walking our way back through table 3.1, the λ–term should be of type $(P \to Q) \to P \to Q$, since it takes a proof of $P \to Q$ and a proof of P and produces a proof of Q. Also, we can inhabitate the type using the fact that applying a term of type P to a term of type $(P \to Q)$ yields a term of type Q by App in the typed λ-calculus. The rule MP corresponds to the following typed term:

$$\vdash \lambda f.\lambda x.\, f\, x : (P \to Q) \to P \to Q$$

Step-by-step β-reduction:

$$(\lambda f.\lambda x.\, f\, x)\ f\ x \ \rightsquigarrow_\beta\ (\lambda x.\, f\, x)\ x \ \rightsquigarrow_\beta\ f\, x$$

This reduction might not say much by itself, so let us annotate it with the most general type of each of the terms in the reduction:

$$\begin{aligned}
&(\lambda f : P \to Q.\lambda x : P.\, f\, x)\ \ f : P \to Q\ \ x : P && \vdash (P \to Q) \to P \to Q \\
&\rightsquigarrow_\beta\ \ (\lambda x : P.\, f\, x)\ \ x : P && \vdash P \to Q \\
&\rightsquigarrow_\beta\ \ f\, x && \vdash Q
\end{aligned}$$

Chapter 4

Categories at Work

Category Theory abstracts the notion of internal structure and studies mathematical objects indirectly. Morphisms are the central objects of study, compositionality and abstraction drive the categorical thought process. In this context, constructions are studied though their relations to one another, instead of from a definitional stand point. This keeps us away from manipulating objects directly, instead, encourages us to apply principles of universality to pinpoint specific properties by uniqueness of morphisms.

It also offers extreme generality and has great power to unify seemingly unrelated fields. It can serve to create blueprints that describe mathematical environments and their properties, and to give structure to patterns that repeat within different fields of mathematics. In some instances, category theory serves as a medium for transporting statements between domains. A categorical formulation some result can often be translated to other context via functors or equivalences of categories.

One field that benefits from practical applications of categorical reasoning is computer science. In particular, categories and type theory are closely intertwined through the notion of compositionality: in category theory, composition is a fundamental structural principle, whereas in type theory, compositionality ensures that complex terms and proofs are constructed in a manner that preserves their meaning.

4.1 What are Categories?

For our purposes, categories arise as a tool to study compositionality, which is why its subject of study are morphisms and not objects. From this point of view, defining what a category is should not result unfamiliar:

Definition 4.1.1. A category $\mathscr{C}$ consists of:

- A collection of objects $\mathrm{Ob}(\mathscr{C})$
- A collection of arrows or morphisims where each arrow f has a domain object $\mathrm{dom}(f)$ and a codomain object $\mathrm{cod}(f)$. These can be defined using $f : A \to B$ to signify $\mathrm{dom}(f) = A$ and $\mathrm{cod}(f) = B$
- A composition operator assigning to each pair of arrows f and g, with $cod(f) = dom(g)$, a composite arrow $g \circ f : \mathrm{dom}(f) \to \mathrm{cod}(g)$ satisfying the following associative law:

  $$h \circ (g \circ f) = (h \circ g) \circ f$$

  where $f : A \to B, g : B \to C, h : C \to D$ are arrows of a category
- For each object A, an identity arrow $id_A : A \to A$ satisfying the following identity law:

  $$\mathrm{id_B} \circ f = f \quad f \circ \mathrm{id_A} = f$$

  for any $f : A \to B$

Remark 25. Literature on the topic usually differentiates between small and large categories. Small categories are those whose collection of objects and whose collection of arrows are both sets. Large categories allow for collections of objects or arrows that are proper classes rather than sets. We will only be concerned with small categories.

Example 4.1.1. Various mathematical constructs fit the description of a category. For example, the category **Set** consists of sets as objects and functions as morphisms, where composition is ordinary function composition and identities are identity functions. Another example is the category **Grp**, whose objects are groups and whose morphisms are group homomorphisms.

Proposition 4.1.1. **Set** is a category where objects correspond to sets $A, B, C, \ldots$ and arrows to functions. For $f : A \to B$ and $g : B \to C$, composition of functions is $(g \circ f)(a) = g(f(a))$, which is a function $g \circ f : A \to C$. Composition is associative since for $h : C \to D$ we have that: $h \circ (g \circ f)(a) = h(g(f(a))) = (h \circ g)(f(a)) = (h \circ g) \circ f(a)$. Lastly, each set A, the identity morphism is $\mathrm{id}_A(a) = a$, satisfying $f \circ \mathrm{id}_A = f = \mathrm{id}_B \circ f$. Therefore, **Set** is a category. □

Hom-set

Take any category $\mathscr{C}$, note how from any given objects $a, b \in \mathscr{C}$, in the general case, there potentially exist several different morphisms. Hom-sets are nothing but the set of all morphisms that can relate any two given objects in a category.

Definition 4.1.2. In a category $\mathscr{C}$, for any two objects $A, B \in \mathrm{Ob}(\mathscr{C})$, the hom-set from A to B is the set of all morphisms from A to B. This set is denoted by $\mathrm{Hom}(A, B)$.

$$\mathrm{Hom}(A, B) = \{f \mid f\colon A \to B \text{ is a morphism in } \mathscr{C}\}$$

We use sets since we deal only with small categories.

Example 4.1.2. Let $\mathscr{C}$ denote a category, consisting of a set of objects, $\mathrm{Ob}(\mathscr{C})$, and a set of arrows, $\mathrm{Hom}(\mathscr{C})$, satisfying the category axioms.

- **empty category**, denoted by 0, is the category with no objects and no morphisms.

$$\mathrm{Ob}(0) = \emptyset, \quad \mathrm{Hom}(0) = \emptyset$$

- **point category**, denoted by 1, is the category with a single object and a single morphism, which must be the identity morphism on that object.

$$\mathrm{Ob}(1) = \{*\}, \quad \mathrm{Hom}(1) = \{\mathrm{id}_*\}$$

  where $\mathrm{id}_* : * \to *$ is the identity morphism on $*$.

- **category 2**, often representing the poset $A \leq B$, is a category with two objects and three morphisms. Formally,

$$\mathrm{Ob}(2) = \{A, B\}, \quad \mathrm{Hom}(2) = \{\mathrm{id}_A, \mathrm{id}_B, f\}$$

  where $\mathrm{id}_A : A \to A$ and $\mathrm{id}_B : B \to B$ are identity morphisms, and $f : A \to B$ is a unique non-identity morphism. The compositions are defined as $f \circ \mathrm{id}_A = f$ and $\mathrm{id}_B \circ f = f$.

Definition 4.1.3. A directed graph consists of a collection of vertices and a collection of edges, where each edge has a designated source vertex and target vertex.

Example 4.1.3. Using a directed graph we can represent the categories 1 and 2 given in 4.1.2, as well as category 3 and further. It goes without saying that we won't use directed graphs to represent the 0 category.

$$\begin{tikzcd}
* \arrow[loop right, "id_*"] & & & B \arrow[loop right, "id_B"] \arrow[d] \\
id_A \arrow[loop left] & A \arrow[r] & B \arrow[loop right, "id_B"] & id_C \arrow[loop left] & A \arrow[r] \arrow[ur] & C \arrow[loop right, "id_C"]
\end{tikzcd}$$

Proposition 4.1.2. Any partially ordered set $(P, \leq)$ can be viewed as a category **Pos** whose objects are elements of P and there is a unique arrow $p \to q$ if and only if $p \leq q$. Composition is defined by transitivity: if $p \leq q$ and $q \leq r$, then $p \leq r$, so the composite $p \to r$ exists and is unique. Associativity holds automatically since there is at most one arrow between any two objects. Lastly, each $p \in P$ has an identity arrow $p \to p$ that corresponds to the reflexivity property of partial orders. Therefore, every poset defines a category. □

Example 4.1.4. Let us consider a simple functional programming language with the following types:

$$T ::= \mathrm{Nat} \mid \mathrm{Bool} \mid T \times T \mid T \to T$$

The type system is closed under product and function type constructors. A valid compound type is $(\mathrm{Nat} \times \mathrm{Bool}) \to \mathrm{Nat}$. A categorical interpretation of this scenario reveals types are objects and functions morphisms.

- Identity $\mathrm{id}_A : A \to A$ can be defined as $\mathrm{id}_A = \lambda x{:}A.\, x$, which is well-typed. Identity laws are verified trivially: $\mathrm{id}_B \circ f = f = f \circ \mathrm{id}_A$.
- Compositionality is defined as sequential application of functions that share domain and codomain through $f : A \to B$ and $g : B \to C$ like $g \circ f : A \to C$ by $g \circ f = \lambda x{:}A.\, g(f(x))$, which is well-typed by function application.
- Composition is associative due to β-equivalence in lambda calculus:

$$h \circ (g \circ f) = \lambda x{:}A.h(g(f(x))) = (h \circ g) \circ f$$

Therefore, the language satisfies all axioms of a category.

Example 4.1.5. **Hask** is a category where the objects are Haskell types such as `Int`, `Bool`, `String`. The morphisms are total Haskell functions between those types such as `f::Int->Bool`. For every type `a`, there is an identity morphism `id::a->a` that maps each value to itself. Morphisms can be composed using `(.)::(b->c)->(a->b)->a->c`, meaning the output of one function becomes the input of another. This composition is associative, and the identity morphisms act as neutral elements for composition. These properties types as objects, functions as morphisms, `id` as identity, and `(.)` as composition make Hask a category.

Category	Description
Set	Sets as objects, functions as morphisms.
Grp	Groups and group homomorphisms.
Ring	Rings and ring homomorphisms.
$\mathbf{Mod}_R$	Modules over a ring R and R-linear maps.
$\mathbf{Vect}_k$	Vector spaces over field k and linear maps.
Top	Topological spaces and continuous functions.
Man	Smooth manifolds with smooth maps.
Hilb	Hilbert spaces and bounded linear maps.

Table 4.1: Key mathematical categories.

Category	Description
Mon	Monoids and monoid homomorphisms.
CPO	Complete partial orders and continuous maps.
Dom	Domains as posets with Scott-continuous maps.
Rel	Sets and binary relations as morphisms.
Pfn	Sets and partial functions as morphisms.
Par	Sets and partial maps (generalized morphisms).
$\mathbf{Kl}(T)$	Kleisli category of a monad T on a base category.
$\mathbf{Endo}(\mathbf{Set})$	Endofunctors on the category of sets.

Table 4.2: Key categories in computer science.

Duality

The categories we have built up until now, have been built by matching structures to the definition of category. However, if we look closely at any category we find that by reversing every morphism $f : A \to B$ into $f^{op} : B \to A$, we find ourselves with a newly created category in our hands. That is if we also reverse composition of morphisms.

Definition 4.1.4. For a category $\mathscr{C}$, the opposite or dual category $\mathscr{C}^{op}$ is defined by:

- $\mathrm{Ob}(\mathscr{C}^{op}) = \mathrm{Ob}(\mathscr{C})$
- For $A, B \in \mathrm{Ob}(\mathscr{C})$, $\mathscr{C}^{op}(A, B) = \mathscr{C}(B, A)$
- Composition is reversed: if $f : A \to B$ and $g : B \to C$ in $\mathscr{C}$, then $f^{op} \circ g^{op} = (g \circ f)^{op}$

This reversal allows any property or construction to be mirrored.

4.2 Monos & Epis & Isos

Monomorphisms, epimorphisms, and isomorphisms arise in category theory as abstract counterparts to injective, surjective, and bijective maps, but their definitions rely on morphisms instead of on elements.

Definition 4.2.1. Let $g_1, g_2 : X \to A$ be morphisms in a category $\mathscr{C}$. A morphism $f : A \to B$ is monic if

$$f \circ g_1 = f \circ g_2 \implies g_1 = g_2 \qquad X \underset{g_2}{\overset{g_1}{\rightrightarrows}} A \xrightarrow{f} B$$

In other words, f is left-cancellable.

Definition 4.2.2. Let $h_1, h_2 : B \to Y$ be morphisms in a category $\mathscr{C}$. A morphism $f : A \to B$ is epic if

$$h_1 \circ f = h_2 \circ f \implies h_1 = h_2 \qquad A \xrightarrow{f} B \underset{h_2}{\overset{h_1}{\rightrightarrows}} Y$$

In other words, f is right-cancellable.

Note 15. Monomorphisims generalize the notion of injectivity, while epimorphisms are the generalization of surjectivity.

Proposition 4.2.1. Given $\mathscr{C}$, monomorphisms in $\mathscr{C}$ are dual to epimorphisms in $\mathscr{C}^{op}$.

Proof. Let $\mathscr{C}$ be a category and $g_1, g_2 : X \to A, f : A \to B$, the dual category $\mathscr{C}^{op}$ has $g_1^{op}, g_2^{op} : A \to X, f^{op} : B \to A$ and so:

$$f \circ g_1 = f \circ g_2 \implies g_1 = g_2$$

in the dual category becomes

$$g_1^{op} \circ f^{op} = g_2^{op} \circ f^{op} \implies g_1^{op} = g_2^{op}$$

which is exactly the definition of epi in $\mathscr{C}$. It is equally easy to check that the converse is also true. □

Definition 4.2.3. An object A is isomorphic to B in a category $\mathscr{C}$ if there exists a morphism $f : A \to B$ such that there exists an inverse morphism $g : B \to A$ with

$$g \circ f = \mathrm{id}_A \quad \text{and} \quad f \circ g = \mathrm{id}_B.$$

Denoted $A \cong B$.

Proposition 4.2.2. Let $\mathscr{C}$ be a category and $f : A \to B$ a morphism in $\mathscr{C}$. If f is an isomorphism in $\mathscr{C}$, then f is also an isomorphism in the opposite category $\mathscr{C}^{\mathrm{op}}$.

Proof. Since f is an isomorphism in $\mathscr{C}$, there exists a morphism $g : B \to A$ such that

$$g \circ f = \mathrm{id}_A \quad \text{and} \quad f \circ g = \mathrm{id}_B.$$

In $\mathscr{C}^{\mathrm{op}}$, the morphisms f and g are reversed: $f^{\mathrm{op}} : B \to A$ and $g^{\mathrm{op}} : A \to B$. The compositions in $\mathscr{C}^{\mathrm{op}}$ satisfy

$$f^{\mathrm{op}} \circ g^{\mathrm{op}} = (g \circ f)^{\mathrm{op}} = \mathrm{id}_A, \quad g^{\mathrm{op}} \circ f^{\mathrm{op}} = (f \circ g)^{\mathrm{op}} = \mathrm{id}_B.$$

Hence f^{op} is an isomorphism in $\mathscr{C}^{\mathrm{op}}$. □

Isomorphisms are fundamental because they formalize the idea of structural equivalence between objects. As explained, Category Theory is mostly concerned with morphisms, which is why, isomorphisms play an important role. Many of the constructions we will introduce, such as products, coproducts, are unique only up to isomorphism, meaning, that they are unable to tell isomorphic objects apart from one another. This is one of the keys to the level of abstraction that Category Theory provides, only reaching up to isomorphism.

4.3 Functors

Category Theory takes itself very seriously. Hence, it is natural to consider functors, that are themselves morphisms in the category of categories. In this way, category theory can study itself. From this perspective, the definition of a functor arises naturally as a pair of functions that map both objects and arrows from one category to another while preserving identities and composition.

Definition 4.3.1. Let $\mathscr{C}$ and $\mathscr{D}$ be categories. A functor $F : \mathscr{C} \to \mathscr{D}$ consists of:

- A function that maps objects from $\mathscr{C}$ to $\mathscr{D}$:

$$\begin{aligned} F : \mathrm{Ob}(\mathscr{C}) &\longrightarrow \mathrm{Ob}(\mathscr{D}) \\ X &\longmapsto F(X) \end{aligned}$$

- A function that maps arroes in $\mathscr{C}$ to $\mathscr{D}$:

$$\begin{aligned} F : \mathrm{Hom}_{\mathscr{C}}(X, Y) &\longrightarrow \mathrm{Hom}_{\mathscr{D}}(F(X), F(Y)) \\ f &\longmapsto F(f) \end{aligned}$$

such that the properties of a category are kept across the transformation:

- For every object $X \in \mathscr{C}$, $F(\mathrm{id}_X) = \mathrm{id}_{F(X)}$.
- For all composable morphisms $f : X \to Y$ and $g : Y \to Z$ in $\mathscr{C}$,

$$F(g \circ f) = F(g) \circ F(f)$$

The following diagram elegantly summarizes the latter definition:

$$\begin{tikzcd} X \arrow[r, "f"] \arrow[d, mapsto, "F"'] & Y \arrow[d, mapsto, "F"] \\ F(X) \arrow[r, "F(f)"'] & F(Y) \end{tikzcd}$$

Example 4.3.1. Haskell captures the notion of functor using the following type class:

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
    (<$) :: a -> f b -> f a
    (<$) = fmap . const
```

There exist several instances for this class, among which we find:

```
instance Functor [] where
    fmap = map
```

That allows us to perform mapping operations on lists like:

```
ghci> fmap (*2) [1,2,3]
[2,4,6]
```

Example 4.3.2. A reformulation from example 2.1.2 in Pierce 1991 that defines a functor called List from the category **Set** → **Set** to illustrate functors from a programming perspective.

- For every set S, we define a new set:

$$\mathrm{List}(S) := \{L \mid L \text{ is a finite list of belements of } S\}$$

  Where the order of elements matters, and duplicates are allowed.

- For every function $f : S \to S'$, we define a new function

$$\begin{aligned} \mathrm{List}(f) &: \mathrm{List}(S) \to \mathrm{List}(S') \\ \mathrm{List}(f)([s_1, s_2, \ldots, s_n]) &:= [f(s_1), f(s_2), \ldots, f(s_n)]. \end{aligned}$$

  So, say for instance we have $S = \{1,2\}$ and $f(s) = 2 \cdot s$. Then,

$$S' = \{2,4\}, \mathrm{List}(\{1,2\}) = \{[], [1], [2], [1,1], [1,2], [2,1], [2,2], [1,1,1], \ldots\}$$

  and

$$\mathrm{List}(f)(\{2,4\}) = \{[], [2], [4], [2,2], [2,4], [4,2], [4,4], [2,2,2], \ldots\}$$

To confirm that List is a functor, it must satisfy the following two laws:

- Identity: For every set S and every list $L \in \mathrm{List}(S)$: $\mathrm{List}(\mathrm{id}_S)(L) = L$.
- Composition: For any two functions $f : S \to S'$ and $g : S' \to S''$, and any list $L \in \mathrm{List}(S)$: $\mathrm{List}(g \circ f)(L) = (\mathrm{List}(g) \circ \mathrm{List}(f))(L)$.

Note 16. $(\mathrm{List}(S), ++, [])$ is a monoid, where $++$ is associative list concatenation and $[]$ is the identity, satisfying $[]++L = L = L++[]$.

We have so far used directed graphs to informally represent categories, now, we leverage functors to introduce a purpose-specific form of directed graph tailored to our setting. We refer to this fine-tuned graphs as diagrams, we emphasize commutative diagrams, those where all paths from any given object to any other are equivalent.

Definition 4.3.2. A diagram in category theory is a functor $F : D \to \mathscr{C}$ where D is the indexing category, that describes the diagram's shape, and $\mathscr{C}$ is the target category. Each object of D is mapped to an object of $\mathscr{C}$, and each morphism of D is mapped to a morphism of $\mathscr{C}$, preserving composition and identities.

Example 4.3.3. Let W, X, Y, Z be objects in a category $\mathscr{C}$ with morphisms

$$f : W \to X, \quad g : W \to Y, \quad h : X \to Z, \quad k : Y \to Z, \quad m : W \to Z.$$

The following diagram illustrates these morphisms:

$$\begin{tikzcd} W \arrow[r, "f"] \arrow[d, "g"'] \arrow[dr, dashed, "m"] & X \arrow[d, "h"] \\ Y \arrow[r, "k"'] & Z \end{tikzcd}$$

This diagram commutes if

$$m = h \circ f = k \circ g,$$

so all three paths from W to Z are equal. For a diagram to commute, this property should hold for any pair of objects.

Example 4.3.4. Consider the categories of groups **Grp** and sets **Set**. We define the functor $U : \mathbf{Grp} \longrightarrow \mathbf{Set}$:

- On objects: for each group $G \in \mathbf{Grp}$, set $U(G)$ to be the set of elements of G.
- On morphisms: for each group homomorphism $f : G \to H$, we define $U(f) = f$, now considered as a function $U(G) \to U(H)$ between sets.

This mapping preserves identities and composition. Diagrammatically, we can represent the action of U on a morphism $f : G \to H$ as:

$$\begin{tikzcd} G \arrow[r, "f"] \arrow[d, mapsto, "U"'] & H \arrow[d, mapsto, "U"] \\ U(G) \arrow[r, "U(f)"'] & U(H) \end{tikzcd}$$

Since this functor "forgets" the structure its domain category we refer to it as forgetful functor.

Contravariant Functors

In category theory, the notion of a functor is introduced as a structure-preserving map between categories. When first defined, a functor is understood to preserve both the direction of morphisms and the way in which they compose. Since functors preserve the the direction of arrows, they are usually called covariant functors. This terminology only becomes relevant once one contrasts them with contravariant functors, which reverse the direction of arrows.

Definition 4.3.3. Let $\mathscr{C}$ and $\mathscr{D}$ be categories. A contravariant functor $F : \mathscr{C} \to \mathscr{D}$ consists of:

- A function that maps objects from $\mathscr{C}$ to $\mathscr{D}$, same as 4.3.1:
- A function that maps arrows in $\mathscr{C}$ to arrows in $\mathscr{D}$, but reverses their direction:

$$\begin{aligned} F : \mathrm{Hom}_{\mathscr{C}}(X, Y) &\longrightarrow \underline{\mathrm{Hom}_{\mathscr{D}}(F(Y), F(X))} \\ f &\longmapsto F(f) \end{aligned}$$

such that identity and composition are preserved, same as 4.3.1. The following diagram summarizes how a contravariant functor reverses arrows:

$$\begin{tikzcd} X \arrow[r, "f"] \arrow[d, mapsto, "F"'] & Y \arrow[d, mapsto, "F"] \\ F(X) & F(Y) \arrow[l, "F(f)"'] \end{tikzcd}$$

The reason for distinguishing between covariant and contravariant functors can be clarified through duality. For any category $\mathscr{D}$, its opposite category $\mathscr{D}^{\mathrm{op}}$ has the same objects as $\mathscr{D}$, but with all morphisms reversed, such that: $\mathrm{Hom}_{\mathscr{D}^{\mathrm{op}}}(A, B) = \mathrm{Hom}_{\mathscr{D}}(B, A)$.

Proposition 4.3.1. Every covariant functor $F : \mathscr{C} \to \mathscr{D}$ can equally well be regarded as a contravariant functor $F : \mathscr{C} \to \mathscr{D}^{\mathrm{op}}$.

Proof. Suppose $F : \mathscr{C} \to \mathscr{D}$ is a covariant functor. Define a functor $F' : \mathscr{C} \to \mathscr{D}^{\mathrm{op}}$ by:

- On objects: $F'(A) := F(A)$ for all $A \in \mathscr{C}$.
- On morphisms: for $f : A \to B$ in $\mathscr{C}$, set $F'(f) := F(f) \in \mathrm{Hom}_{\mathscr{D}}(F(A), F(B))$. But in $\mathscr{D}^{\mathrm{op}}$ we have $\mathrm{Hom}_{\mathscr{D}}(F(A), F(B)) = \mathrm{Hom}_{\mathscr{D}^{\mathrm{op}}}(F(A), F(B))$, so $F'(f)$ is indeed a morphism in $\mathscr{D}^{\mathrm{op}}$.

Functoriality of F' follows immediately from functoriality of F, since identities and composition are preserved in $\mathscr{D}^{\mathrm{op}}$ exactly as in $\mathscr{D}$, but with reversed notation. Hence F' is a contravariant functor into $\mathscr{D}^{\mathrm{op}}$. □

Remark 26. Thus, what we call a *"covariant functor"* is just a functor $\mathscr{C} \to \mathscr{D}$, while a *"contravariant functor"* from $\mathscr{C}$ to $\mathscr{D}$ is a functor $\mathscr{C} \to \mathscr{D}^{\mathrm{op}}$. In this sense, the terminology covariant vs. contravariant simply reflects whether the functor sends morphisms into the target category itself or into its dual.

Example 4.3.5.

4.4 Natural Transformations

To define natural transformations we will apply the same reasoning scheme as we did with functors. We had categories and we defined functors as morphisms between categories, we do the same with natural transformations, that are nothing but morphisms from functors to functors. Suppose we have two functors $F, G : \mathscr{C} \to \mathscr{D}$. These represent two distinct ways of translating $\mathscr{C}$ into $\mathscr{D}$. A natural transformation between F and G then provides a systematic way to relate their outputs.

Definition 4.4.1. Let $\mathscr{C}$ and $\mathscr{D}$ be categories, and let $F, G : \mathscr{C} \to \mathscr{D}$ be functors. A natural transformation $\alpha : F \Rightarrow G$ between them:

$$\begin{tikzcd} \mathscr{C} \arrow[r, bend left=60, "F"] \arrow[r, bend right=60, "G"'] \arrow[r, phantom, "\Downarrow\alpha"] & \mathscr{D} \end{tikzcd}$$

is an assignment to every object $X \in \mathrm{Ob}(\mathscr{C})$ of a morphism $\alpha_X : F(X) \to G(X)$ in $\mathscr{D}$ called the component of α at X, such that for every morphism $f : X \to Y$ in $\mathscr{C}$, the following diagram commutes:

$$\begin{tikzcd} F(X) \arrow[r, "\alpha_X"] \arrow[d, "F(f)"'] & G(X) \arrow[d, "G(f)"] \\ F(Y) \arrow[r, "\alpha_Y"] & G(Y) \end{tikzcd}$$

An example from the `ghci` prompt that shows `reverse` is natural and preserves commutativity:

```
ghci> (reverse . (fmap toUpper)) "abc"
"CBA"
ghci> ((fmap toUpper) . reverse) "abc"
"CBA"
```

Definition 4.4.2. An endofunctor on a category $\mathscr{C}$ is a functor $F : \mathscr{C} \to \mathscr{C}$ that maps objects and morphisms of $\mathscr{C}$ to objects and morphisms of the same category, preserving identities and composition.

Example 4.4.1. A custom instantiation of functor similar to the previous example:

```
data BinTree a
    = Empty
    | Node a (BinTree a) (BinTree a)
    deriving (Show, Eq)

instance Functor BinTree where
    fmap _ Empty = Empty
    fmap f (Node x l r) = Node (f x) (fmap f l) (fmap f r)

reversePairs :: BinTree a -> BinTree a
reversePairs Empty = Empty
reversePairs (Node x l r) = Node x (reversePairs r) (reversePairs l)

testNaturality :: (Eq b) => (a -> b) -> BinTree a -> Bool
testNaturality f t =
    (reversePairs . fmap f) t == (fmap f . reversePairs) t
```

We now can call the `testNaturality` function to assert whether the `f` we provide makes the diagram commute.

```
ghci> f n = 2 * n
ghci> testNaturality f t
True
```

Note 17. From this example we extract a design pattern. For a given data structure whether a graph, a tree or otherwise, an operation that modifies structure can be considered a natural transform whenever we only deal with functions that don't modify structure.

In this case, any function that just operates on the keys will be a valid operation since it does not alter structure. Note how `testNaturality` just test naturality on a given argument. To support the claim that `reversePairs` it is in fact a natural transformation we proceed using structural induction.

Theorem 4.4.1. Let `RP`, `No` and `Em` represent `reversePairs`, `Node` and `Empty` respectively. The function $\mathtt{RP} : \mathtt{BinTree} \to \mathtt{BinTree}$ defined by $\mathtt{RP}(\mathtt{Em}) = \mathtt{Em}$, $\mathtt{RP}(\mathtt{No}\ x\ l\ r) = \mathtt{No}\ x\ (\mathtt{RP}(r))\ (\mathtt{RP}(l))$ is a natural transformation. That is, for all functions $f : A \to B$ and trees $t : \mathtt{BinTree}\ A$, $\mathtt{fmap}\ f\ (\mathtt{RP}(t)) = \mathtt{RP}(\mathtt{fmap}\ f\ t)$.

Proof. We prove by structural induction on t.

- Base case: $t = \mathtt{Em}$. Then $\mathtt{fmap}\ f\ (\mathtt{RP}(\mathtt{Em})) = \mathtt{fmap}\ f\ (\mathtt{Em}) = \mathtt{Em} = \mathtt{RP}(\mathtt{Em}) = \mathtt{RP}(\mathtt{fmap}\ f\ \mathtt{Em})$ and the naturality condition holds.
- Inductive step: assume the naturality condition holds for subtrees l and r of depth $\leq n$. Let $t = \mathtt{No}\ x\ l\ r$. Then $\mathtt{RP}(t) = \mathtt{No}\ x\ (\mathtt{RP}(r))\ (\mathtt{RP}(l))$ and

$$\mathtt{fmap}\ f\ (\mathtt{RP}(t)) = \mathtt{fmap}\ f\ (\mathtt{No}\ x\ (\mathtt{RP}(r))\ (\mathtt{RP}(l))) = \mathtt{No}\ (f x)\ (\mathtt{fmap}\ f\ (\mathtt{RP}(r)))\ (\mathtt{fmap}\ f\ (\mathtt{RP}(l)))$$

By the inductive hypothesis, $\texttt{fmap}\ f\ (\texttt{RP}(l)) = \texttt{RP}(\texttt{fmap}\ f\ l), \texttt{fmap}\ f\ (\texttt{RP}(r)) = \texttt{RP}(\texttt{fmap}\ f\ r)$ so we can write:

$$\texttt{fmap}\ f\ (\texttt{RP}(t)) = \texttt{No}\ (f\,x)\ (\texttt{RP}(\texttt{fmap}\ f\ r))\ (\texttt{RP}(\texttt{fmap}\ f\ l)) = \texttt{RP}(\texttt{No}\ (f\,x)\ (\texttt{fmap}\ f\ l)\ (\texttt{fmap}\ f\ r)) = \texttt{RP}(\texttt{fmap}\ f\ t)$$

Hence the naturality condition holds for t.

By structural induction, RP is a natural transformation. □

Natural transformations, as well as functors can be composed as well, we study now the two ways there are of composing natural transformations:

Remark 27. Given functors $F, G, H : \mathscr{C} \to \mathscr{D}$, natural transformations $\alpha : F \Rightarrow G$ and $\beta : G \Rightarrow H$ can be composed vertically:

$$\beta \circ \alpha : F \Rightarrow H$$

$$\begin{tikzcd}
\mathscr{C} \arrow[r, bend left=60, "F"{name=F}] \arrow[r, "G"'{name=G}] \arrow[r, bend right=60, "H"'{name=H}] & \mathscr{D} \arrow[from=F, to=G, Rightarrow, "\alpha"] \arrow[from=G, to=H, Rightarrow, "\beta"]
\end{tikzcd}
\qquad
\begin{tikzcd}
F(X) \arrow[r, "\alpha_X"] \arrow[d, "F(f)"'] & G(X) \arrow[r, "\beta_X"] \arrow[d, "G(f)"'] & H(X) \arrow[d, "H(f)"] \\
F(Y) \arrow[r, "\alpha_Y"] & G(Y) \arrow[r, "\beta_Y"] & H(Y)
\end{tikzcd}$$

with components $(\beta \circ \alpha)_X = \beta_X \circ \alpha_X$ for each object $X \in \mathrm{Ob}(\mathscr{C})$.

Remark 28. Given categories $\mathscr{C}, \mathscr{D}, \mathscr{E}$, functors $F, G : \mathscr{C} \to \mathscr{D}$ and $J, K : \mathscr{D} \to \mathscr{E}$, and natural transformations $\alpha : F \Rightarrow G$ and $\gamma : J \Rightarrow K$, we can define the horizontal composition:

$$\gamma * \alpha : J \circ F \Rightarrow K \circ G$$

$$\begin{tikzcd}
\mathscr{C} \arrow[r, bend left=60, "F"{name=F}] \arrow[r, "G"'{name=G}] & \mathscr{D} \arrow[r, bend left=60, "J"{name=J}] \arrow[r, "K"'{name=K}] & \mathscr{E} \arrow[from=F, to=G, Rightarrow, "\alpha"] \arrow[from=J, to=K, Rightarrow, "\gamma"]
\end{tikzcd}
\qquad
\begin{tikzcd}
J(F(X)) \arrow[r, "\gamma_{F(X)}"] \arrow[d, "J(\alpha_X)"'] & K(F(X)) \arrow[d, "K(\alpha_X)"] \\
J(G(X)) \arrow[r, "\gamma_{G(X)}"] & K(G(X))
\end{tikzcd}$$

with components $(\gamma * \alpha)_X = \gamma_{G(X)} \circ J(\alpha_X) = K(\alpha_X) \circ \gamma_{F(X)}$ for each $X \in \mathrm{Ob}(\mathscr{C})$.

Remark 29. The collection of all functors $F : \mathscr{C} \to \mathscr{D}$ together with natural transformations between them forms the functor category $\mathbf{Fun}(\mathscr{C}, \mathscr{D}) \in \mathrm{Ob}(\mathbf{Cat})$.

Example 4.4.2. This example builds the natural transform `reversePairs` from the composition of two variants of reverse pairs, one that reverses even depths, and another that does the converse on odd depths.

```
reverseDepth :: BinTree a -> Int -> BinTree a
reverseDepth Empty _ = Empty
reverseDepth (Node x l r) d
    | even d    = Node x (reverseDepth r (d+1)) (reverseDepth l (d+1))
    | otherwise = Node x (reverseDepth l (d+1)) (reverseDepth r (d+1))

reverseEven :: BinTree a -> BinTree a
reverseEven t = reverseDepth t 0

reverseOdd :: BinTree a -> BinTree a
reverseOdd t = reverseDepth t 1

horizReversePairs :: BinTree a -> BinTree a
horizReversePairs = reverseEven . reverseOdd

testHorizNaturality :: (Eq b) => (a -> b) -> BinTree a -> Bool
testHorizNaturality f t =
    (horizReversePairs . fmap f) t == (fmap f . horizReversePairs) t
```

Theorem 4.4.2. Let $\texttt{RE} = \texttt{reverseEven}$, $\texttt{RO} = \texttt{reverseOdd}$, and $\texttt{RP} = \texttt{reversePairs}$, and let Em and No denote Empty and Node, respectively. Then for any binary tree t, $\texttt{RP}(t) = (\texttt{RO} * \texttt{RE})(t)$.

Proof. We prove the statement by structural induction on t.

- Base case:
  - If $t = \texttt{Em}$, then $\texttt{RE}(\texttt{Em}) = \texttt{RO}(\texttt{Em}) = \texttt{RP}(\texttt{Em}) = \texttt{Em}$.
  - If $t = \texttt{No}\ x\ \texttt{Em}\ \texttt{Em}$, then $\texttt{RE}(t) = \texttt{RO}(t) = \texttt{RP}(t) = \texttt{No}\ x\ \texttt{Em}\ \texttt{Em}$.
- Inductive step: Assume that for all subtrees of depth $\leq n$, the equality $\texttt{RP}(s) = (\texttt{RO} * \texttt{RE})(s)$ holds. Let $t = \texttt{No}\ x\ L\ R$ be a tree of depth $n+1$.
  - Applying RE gives $\texttt{RE}(t) = \texttt{No}\ x\ (\texttt{RE}(R))\ (\texttt{RE}(L))$.
  - Applying RO to $\texttt{RE}(t)$ gives $\texttt{RO}(\texttt{RE}(t)) = \texttt{No}\ x\ (\texttt{RO}(\texttt{RE}(R)))\ (\texttt{RO}(\texttt{RE}(L)))$.
  - By the inductive hypothesis, $\texttt{RO}(\texttt{RE}(L)) = \texttt{RP}(L)$ and $\texttt{RO}(\texttt{RE}(R)) = \texttt{RP}(R)$.
    Hence, $\texttt{RO}(\texttt{RE}(t)) = \texttt{No}\ x\ (\texttt{RP}(R))\ (\texttt{RP}(L)) = \texttt{RP}(t)$.

By structural induction, we conclude that $\texttt{RP} = \texttt{RO} * \texttt{RE}$ for all binary trees t.

□

Corollary 4.4.3. $(\texttt{RO} * \texttt{RE})$ is a natural transformation.

Proof. Direct by 4.4.1 and 4.4.2. □

Remark 30. Compilers can use natural transforms to avoid performing unnecessary work and wasting precious resources. Take for instance the following `safeRoot` function that returns the element at the root of a tree if there is one:

```
safeRoot :: BinTree a -> Maybe (BinTree a)
safeRoot Empty = Nothing
safeRoot (Node x _ _) = Just (Node x Empty Empty)
```

Check, that by naturality, `(safeRoot . fmap f)` achieves the same as `(fmap f . safeRoot)`. So in the case that `f` is particularly expensive, we can apply `(f . safeRoot)` instead of `(f . safeRoot)`. The former more inexpensive because we first get rid of every element other than the root and then perform `f`, as opposed to performing `f` on every single node of the tree.

4.5 Universal Constructions

Universality is a central concept in category theory, capturing the idea that certain constructions are optimal or canonical in a precise sense. A universal construction ensures that any relevant morphism factors uniquely through a specific object and arrow. Universal arrows formalize this notion: they express a universal property from an object in one category to a functor into another. Moreover, by their very nature, universal arrows explain why universal properties are unique up to isomorphism.

Definition 4.5.1. Let $S : \mathscr{D} \to \mathscr{C}$ be a functor and let X be an object of $\mathscr{C}$. A universal arrow from X to S is a pair $\langle R, u \rangle$ consisting of an object R of $\mathscr{D}$ and a morphism $u : X \to S(R)$ in $\mathscr{C}$ such that for every object Y of $\mathscr{D}$ and every morphism $f : X \to S(Y)$ in $\mathscr{C}$, there exists a unique morphism $f' : R \to Y$ in $\mathscr{D}$ with $S(f') \circ u = f$.

This condition means that every arrow from X to $S(Y)$ factors uniquely through u, $S(f') \circ u = f$, as shown in the following commutative diagram:

$$\begin{tikzcd} X \arrow[r, "u"] \arrow[dr, "f"'] & S(R) \arrow[d, dashed, "S(f')"] & R \arrow[d, "f'"] \\ & S(Y) & Y \end{tikzcd}$$

Terminal & Initial Objects

Definition 4.5.2. An object T in a category $\mathscr{C}$ is terminal if for every object $X \in \mathrm{Ob}(\mathscr{C})$, there exists a unique morphism $!_X : X \to T$. This property characterizes T up to unique isomorphism.

Proposition 4.5.1. In the category of small categories, **Cat**, the point category 1 is terminal: for every small category $\mathscr{C}$, there exists a unique functor $!_{\mathscr{C}} : \mathscr{C} \to 1$. which sends every object of $\mathscr{C}$ to $*$ and every morphism to id_*.

Proof. Recall that the point category 1 has a single object $*$ and a single morphism $\mathrm{id}_* : * \to *$.

- Existence: Define $!_{\mathscr{C}}$ on objects by $!_{\mathscr{C}}(X) = *$ for every object $X \in \mathrm{Ob}(\mathscr{C})$, and on morphisms by $!_{\mathscr{C}}(f) = \mathrm{id}_*$ for every morphism f of $\mathscr{C}$. This assignment is well defined and preserves identities and composition:

$$!_{\mathscr{C}}(\mathrm{id}_X) = \mathrm{id}_* \quad \text{and} \quad !_{\mathscr{C}}(g \circ f) = \mathrm{id}_* = \mathrm{id}_* \circ \mathrm{id}_* = !_{\mathscr{C}}(g) \circ !_{\mathscr{C}}(f)$$

  so $!_{\mathscr{C}}$ is a functor $\mathscr{C} \to 1$.

- Uniqueness: If $F : \mathscr{C} \to 1$ is any functor, then for each object $X \in \mathscr{C}$ the object $F(X)$ must be an object of 1, hence $F(X) = *$. For each morphism $f : X \to Y$ in $\mathscr{C}$ the morphism $F(f) : * \to *$ must be a morphism of 1, and the only such morphism is id_*, so $F(f) = \mathrm{id}_*$. Thus F coincides with the above $!_{\mathscr{C}}$, proving uniqueness.

Therefore for every small category $\mathscr{C}$ there is a unique functor $\mathscr{C} \to 1$, i.e. 1 is a terminal object in **Cat**. □

Example 4.5.1. Let $\mathscr{C}$ be a category and let $F : 1 \to \mathscr{C}$ be the unique functor from the point category 1. For any object $C \in \mathscr{C}$, a terminal object T with the unique morphism $u : C \to T$ forms a universal arrow from C to F, because for any other morphism $f : C \to T$ (there is only one, by definition of terminal object), the identity morphism $\mathrm{id}_T : T \to T$ is the unique morphism making the diagram commute: $\mathrm{id}_T \circ u = f$. This shows that terminal objects are precisely universal arrows from the identity functor on 1 to $\mathscr{C}$.

Example 4.5.2. The following Haskell snippet provides a feasible definition for terminal object, where arguments of a given type `a`, are all collapsed to `()`.

```
terminate :: a -> ()
terminate _ = ()

ghci> terminate "I'm on death row :'("
()
```

Proposition 4.5.2. Let $\mathscr{C}$ be a category. Any two terminal objects of $\mathscr{C}$ are isomorphic. More precisely, if T and T' are terminal objects then there exists a unique isomorphism $i : T \to T'$, whose inverse is the unique map $T' \to T$.

Proof. Let T and T' be terminal objects of $\mathscr{C}$. By terminality of T' there is a unique morphism $i : T \to T'$, and by terminality of T there is a unique morphism $j : T' \to T$. Consider the composite $j \circ i : T \to T$. Since T is terminal, the identity id_T is the unique morphism $T \to T$, hence $j \circ i = \mathrm{id}_T$. Similarly, the composite $i \circ j : T' \to T'$ must equal $\mathrm{id}_{T'}$ because T' is terminal. Therefore i and j are mutual inverses, so i is an isomorphism with inverse j. Uniqueness: if $i' : T \to T'$ were any isomorphism, then by terminality of T' we must have $i' = i$, so the isomorphism is unique. □

Definition 4.5.3. By duality, initial objects are dual to terminal objects. An object I in a category $\mathscr{C}$ is initial if for every object $X \in \mathrm{Ob}(\mathscr{C})$, there exists a unique morphism $!_X : I \to X$ Dually, the point category 1 serves as an initial object in the opposite category $\mathbf{Cat}^{\mathrm{op}}$. Concretely, in **Cat** itself, there is also a unique functor $1 \to \mathscr{C}$ for any small category $\mathscr{C}$, sending $*$ to any chosen object of $\mathscr{C}$ and id_* to the identity morphism on that object.

Products and Coproducts

When we hear the word product, it is natural to think of the Cartesian product of sets: ordered pairs $\langle a, b\rangle$ with corresponding projections π_1, π_2. That is fine intuition in the category **Set**, but category theory is not about carrying over constructions element-by-element. Instead, it asks us to step back and characterize objects purely in terms of how morphisms interact with them.

So, when we speak of products, and later coproducts, in a category, we are not importing the set-theoretic construction. We are giving a new definition that says: the product is whatever object plays the right role with respect to morphisms. This shift in perspective is crucial: it frees us from thinking in terms of elements and lets the same idea apply in categories where "elements" do not make sense at all.

Definition 4.5.4. Let $\mathscr{C}$ be a category and let $A, B \in \mathrm{Ob}(\mathscr{C})$. A product of A and B is an object $A \times B$ together with morphisms $\pi_1 : A \times B \to A, \pi_2 : A \times B \to B$ such that for every object X with morphisms $f : X \to A$ and $g : X \to B$, there exists a unique morphism $\langle f, g\rangle : X \to A \times B$ making the diagram commute:

$$\begin{tikzcd} & X \arrow[dl, "f"'] \arrow[d, dashed, "\langle f,g\rangle"] \arrow[dr, "g"] & \\ A & A \times B \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & B \end{tikzcd}$$

$$\pi_1 \circ \langle f, g\rangle = f \quad \pi_2 \circ \langle f, g\rangle = g$$

Note 18. In these diagrams, the dashed arrow indicates a unique morphism guaranteed by the universal property of the product or coproduct. That is, for any mediating maps as in the diagram, there exists exactly one morphism making the diagram commute.

Proposition 4.5.3. From Imada 2019: In the category **Set**, the product of objects A and B is their Cartesian product $A \times B$.

Proof. Let X be a set and let $x_1 : X \to A$ and $x_2 : X \to B$. Consider the function

$$\begin{aligned} \langle x_1, x_2\rangle : X &\to A \times B \\ x &\mapsto (x_1(x), x_2(x)) \end{aligned}$$

By definition, we have $x_1 = \pi_1 \circ \langle x_1, x_2\rangle$ and $x_2 = \pi_2 \circ \langle x_1, x_2\rangle$. To show uniqueness of $\langle x_1, x_2\rangle$, suppose there is a morphism $f : X \to A \times B$ such that $x_1 = \pi_1 \circ f$ and $x_2 = \pi_2 \circ f$. Take $x \in X$ and denote

$$\langle x_1, x_2\rangle(x) = (a, b), \qquad f(x) = (a', b').$$

Then $a = \pi_1((a, b)) = \pi_1(f(x)) = x_1(x) = \pi_1(\langle x_1, x_2\rangle(x)) = \pi_1((a', b')) = a'$, and similarly $b = b'$. Hence $(a, b) = (a', b')$, so $f = \langle x_1, x_2\rangle$. Since (X, x_1, x_2) was arbitrary, it follows that $(A \times B, \pi_1, \pi_2)$ is the product of A and B in **Set**. □

Remark 31. From Imada 2019: Products are not unique. For example, in **Set** both $(A \times B, \pi_1, \pi_2)$ and $(B \times A, \pi_2, \pi_1)$ are products of A and B. However, there is a bijection $A \times B \to B \times A$, given by $(a, b) \mapsto (b, a)$. This means that products are unique up to isomorphism. In fact, in any category, every product of two objects is unique up to a unique isomorphism.

Definition 4.5.5. Let $\mathscr{C}$ be a category and let $A, B \in \mathrm{Ob}(\mathscr{C})$. A coproduct of A and B is an object $A + B$ together with morphisms $\iota_1 : A \to A + B, \iota_2 : B \to A + B$ such that for every object X with morphisms $f : A \to X$ and $g : B \to X$, there exists a unique morphism $[f, g] : A + B \to X$ making the diagram commute:

$$\begin{tikzcd} A \arrow[r, "\iota_1"] \arrow[dr, "f"'] & A + B \arrow[d, dashed, "{[f,g]}"] & B \arrow[l, "\iota_2"'] \arrow[dl, "g"] \\ & X & \end{tikzcd}$$

$$[f, g] \circ \iota_1 = f \quad [f, g] \circ \iota_2 = g$$

Remark 32. Coproducts are dual to products.

Exponentials

Definition 4.5.6. Let $\mathscr{C}$ be a category with finite products. For objects $A, B \in \mathscr{C}$, an exponential object B^A is an object together with an evaluation morphism

$$\mathrm{ev}_{A,B} : B^A \times A \longrightarrow B$$

such that for every object $C \in \mathscr{C}$ and every morphism $f : C \times A \longrightarrow B$ there exists a unique morphism $\lambda f : C \longrightarrow B^A$ making the following diagram commute:

$$\begin{tikzcd} C \times A \arrow[rr, dashed, "\lambda f \times 1_A"] \arrow[dr, "f"'] & & B^A \times A \arrow[dl, "\mathrm{ev}_{A,B}"] \\ & B & \end{tikzcd}$$

That is, $\mathrm{ev}_{A,B} \circ (\lambda f \times 1_A) = f$.

Proposition 4.5.4. The exponential object B^A, if it exists, is unique up to unique isomorphism.

The universal constructions we have studied so far—products, coproducts, terminal objects, and exponentials—are by no means the only ones of importance in category theory. Other central constructions include equalizers and coequalizers, pullbacks and pushouts, as well as more general limits and colimits. For a broader treatment of these notions, one can consult standard references in category theory, such as *Categories for the Working Mathematician* Lane 1971, *Category Theory for Computing Science* Barr and Wells 1998 and *Basic Category Theory for Computer Scientists* Pierce 1991.

4.6 Closed Cartesian Categories

A cartesian closed category is a category with three key pieces of structure. First, it has a terminal object, which plays the role of a unique trivial object that every other object admits a morphism to. Second, it has products, which capture the idea of combining objects together, with canonical ways of recovering each component. Finally, it has exponentials, which express that the process of mapping from one object to another can itself be represented as an object in the category. In other words, the collection of morphisms from one object to another is not just external data, but has an internal representation.

Definition 4.6.1. A category $\mathscr{C}$ is called cartesian closed or closed cartesian if:

- $\mathscr{C}$ has a terminal object 1 and all binary products $A \times B$.
- for every pair of objects $A, B \in \mathscr{C}$ there exists an object B^A, called the exponential together with an evaluation morphism $\mathrm{ev}_{A,B} : B^A \times A \longrightarrow B$ satisfying the following universal property: for every object C and every morphism $f : C \times A \longrightarrow B$ there is a unique morphism $\lambda f : C \longrightarrow B^A$ called the transpose or curried map such that the triangle below commutes:

$$\begin{tikzcd} C \times A \arrow[r, "f"] \arrow[d, dashed, "\lambda f \times \mathrm{id}_A"'] & B \\ B^A \times A \arrow[ur, "\mathrm{ev}_{A,B}"'] & \end{tikzcd}$$

Thus, a CCC is a setting where not only objects and morphisms exist, but also the very process of mapping between objects is itself represented as an object of the category. In this way, CCCs resemble deduction systems: every morphism can be systematically built from basic constructions, just as every proof is assembled step by step from simpler proofs.

Currying diagram (another view). The curry/uncurry correspondence is usually summarized by the commutative square/triangle above; to illustrate the bijection of hom-sets one often writes the natural isomorphism:

$$\mathrm{Hom}(C \times A, B) \xrightarrow{\cong} \mathrm{Hom}(C, B^A), \qquad f \mapsto \lambda f,$$

with inverse given by $g \mapsto \mathrm{ev} \circ (g \times \mathrm{id}_A)$. This inverse relation is represented by the commuting diagram

$$\begin{tikzcd} C \times A \arrow[r, "\lambda f \times \mathrm{id}_A"] \arrow[dr, "f"'] & B^A \times A \arrow[d, "\mathrm{ev}"] \\ & B \end{tikzcd}$$

and the uniqueness part of the universal property ensures $\lambda(\mathrm{ev} \circ (g \times \mathrm{id}_A)) = g$ and $\mathrm{ev} \circ (\lambda f \times \mathrm{id}_A) = f$.

Adjunctions

The universal constructions we covered all fall under the umbrella of adjoint functors.

Definition 4.6.2 (Adjunction)**.** Let $\mathscr{C}$ and $\mathscr{D}$ be categories. An *adjunction* between $\mathscr{C}$ and $\mathscr{D}$ consists of a pair of (covariant) functors

$$F : \mathscr{D} \to \mathscr{C} \quad \text{and} \quad G : \mathscr{C} \to \mathscr{D},$$

together with, for every $c \in \mathrm{Ob}(\mathscr{C})$ and $d \in \mathrm{Ob}(\mathscr{D})$, a natural family of bijections

$$\Phi_{c,d} : \mathrm{Hom}_{\mathscr{C}}(Fd, c) \;\cong\; \mathrm{Hom}_{\mathscr{D}}(d, Gc).$$

The naturality condition means that these bijections assemble into natural isomorphisms of functors:

- For fixed $c \in \mathscr{C}$,
$$\mathscr{C}(F-, c) : \mathscr{D} \to \mathbf{Set}^{\mathrm{op}} \quad \cong \quad \mathscr{D}(-, Gc) : \mathscr{D} \to \mathbf{Set}^{\mathrm{op}}.$$
- For fixed $d \in \mathscr{D}$,
$$\mathscr{C}(Fd, -) : \mathscr{C} \to \mathbf{Set} \quad \cong \quad \mathscr{D}(d, G-) : \mathscr{C} \to \mathbf{Set}.$$

The functor F is called the *left adjoint* to G, while G is the *right adjoint* to F, and we write

$$F \dashv G.$$

Adjunctions can be viewed as a weak form of equivalence of categories: every equivalence induces an adjunction, although not every adjunction is an equivalence. In many cases, an adjunction can be upgraded to an equivalence by suitable natural modifications of the categories and functors involved.

Using adjunctions we can formulate alternative definitions for the universal constructions we already covered. To illustrate, we give a concise definition for closed exponential:

Definition 4.6.3. Equivalently, the map $f \mapsto \lambda f$ yields a natural bijection

$$\mathrm{Hom}_{\mathscr{C}}(C \times A, B) \cong \mathrm{Hom}_{\mathscr{C}}(C, B^A)$$

natural in C and B. Saying $\mathscr{C}$ is cartesian closed is equivalent to saying that for every object A the functor $- \times A : \mathscr{C} \to \mathscr{C}$ has a right adjoint $(-)^A : \mathscr{C} \to \mathscr{C}$, i.e.

$$- \times A \dashv (-)^A.$$

References & Resources The categorical content of this chapter is based on foundational works in category theory. Mac Lane Lane 1971 provides the formal structure of categories, functors, and natural transformations; Pierce Pierce 1991 and Barr and Wells Barr and Wells 1998 are used to connect Mac Lane's ideas with computer science. Content related to Lambek is also influenced by Jacobs 1999.

Chapter 5

Curry–Howard–Lambek & Categorical Semantics

Categorical semantics provides a formal, structural interpretation of type systems using the language of category theory. Having introduced typed λ-calculus and the Curry–Howard correspondence, we have seen how types correspond to propositions and terms to proofs. These systems can be viewed as deduction systems, where derivations are constructed according to inference rules, identities correspond to trivial proofs, and composition of derivations reflects sequential application of rules. This perspective emphasizes the compositional nature of proofs: complex derivations are built from simpler ones, and the structure of the system ensures correctness and consistency.

Categories provide a natural generalization of this viewpoint. A category can itself be interpreted as an abstract deduction system in which objects represent propositions or types, morphisms represent proofs or derivations, and composition of morphisms corresponds to chaining proofs, while identity morphisms correspond to reflexive or trivial derivations. Categorical constructions naturally mirror logical operations and type constructors: products correspond to conjunctions, coproducts correspond to disjunctions, and exponentials correspond to implication or function types. In this way, a category encodes the structural relationships between types and proofs, providing a compositional framework that abstracts and extends the reasoning we observed in typed λ-calculus.

This perspective also suggests a natural pathway for further generalization. Just as Curry–Howard relates types and proofs, one can extend the correspondence to Curry–Howard–Lambek (CHL), interpreting typed λ-calculi fully within categorical structures. This extension allows reasoning not only about individual proofs and terms but about entire type systems and their semantic behavior, opening the door to categorical models of richer logical systems and functional programming languages.

5.1 Categories as Deduction Systems

Categories can be viewed as abstract deduction systems where:

- **Objects** correspond to propositions or types.
- **Morphisms** correspond to proofs or derivations: a morphism $f : A \to B$ can be seen as a proof that A entails B.
- **Composition** of morphisms corresponds to composition of proofs: if $f : A \to B$ and $g : B \to C$, then $g \circ f : A \to C$ represents the derived proof.
- **Identity morphisms** correspond to trivial or reflexive proofs (A implies A).

In this view:

- **Products** model conjunction: $A \times B$ represents $A \wedge B$, with projections corresponding to elimination rules.
- **Coproducts** model disjunction: $A + B$ represents $A \vee B$, with injections corresponding to introduction rules.
- **Exponentials** (B^A) model implication: a morphism $f : C \times A \to B$ corresponds to a proof of $A \to B$ under context C.

This is essentially the *Curry–Howard–Lambek correspondence*:

$$\text{Types} \leftrightarrow \text{Propositions}, \quad \text{Terms} \leftrightarrow \text{Proofs}, \quad \text{Category structure} \leftrightarrow \text{Deduction system}.$$

Thus, a category is not just a collection of objects and arrowsit encodes how propositions can be derived from each other in a formal, compositional way.

Proposition 5.1.1. The category of types in the simply typed λ-calculus, with objects as types and morphisms as closed terms modulo $\beta\eta$-equivalence, forms a *Cartesian Closed Category*.

Proof. Define the category $\mathscr{C}(\lambda)$ as follows:

- Objects: Types.
- Morphisms: $A \to B$ are closed terms $c : A \to B$ modulo $\beta\eta$-equivalence.
- Identity: $1_A = \lambda x.x$.
- Composition: $c \circ b = \lambda x.\, c(b\, x)$.

Well-defined category. Unit laws:

$$c \circ 1_B = \lambda x.\, c((\lambda y.y)x) = \lambda x.\, c(x) = c, \quad 1_C \circ c = \lambda x.\, (\lambda y.y)(c\, x) = \lambda x.\, c(x) = c$$

Associativity:

$$c \circ (b \circ a) = \lambda x.\, c((b \circ a)\, x) = \lambda x.\, c(b(a\, x)) = (c \circ b) \circ a.$$

Binary products. For types A, B, define

$$p_1 = \lambda z.\, \mathrm{fst}(z), \quad p_2 = \lambda z.\, \mathrm{snd}(z), \quad (a, b) = \lambda x.\, (a\, x,\, b\, x).$$

Then

$$p_1 \circ (a, b) = a, \quad p_2 \circ (a, b) = b,$$

and for any $c : X \to A \times B$ with $p_1 \circ c = a$ and $p_2 \circ c = b$, we have

$$c = \lambda x.\, (p_1 \circ c\, x,\, p_2 \circ c\, x) = (a, b),$$

so the product is unique.

CCC structure. Function types $A \to B$ and λ-abstraction provide exponentials, completing the CCC structure. □

Table 5.1: The Curry–Howard–Lambek Correspondence

Intuitionistic Logic	**Typed λ-Calculus**	**Category Theory / Lambek**
Proposition A	Type A	Object A
Proof of A	Term of type A	Morphism $f : 1 \to A$
Implication $A \to B$	Function type $A \to B$	Exponential object B^A
Conjunction $A \wedge B$	Product type $A \times B$	Product $A \times B$ and projections
Disjunction $A \vee B$	Sum type $A + B$	Coproduct $A + B$ and injections
$\top$	1	Terminal object 1
$\bot$	0	Initial object 0
Proof normalization	β-reduction	Commutativity of diagrams

5.2 Closing Remarks and Perspectives

Dependent Typing

Dependent Types and Categories

Bibliography

Howard, William Alvin (1980). "The Formulae-as-Types Notion of Construction". In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus, and Formalism*. Ed. by Haskell Curry et al. Originally circulated as a manuscript in 1969. New York: Academic Press, pp. 479–490.

Lambek, Joachim (1980). "From λ-calculus to Cartesian closed categories". In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Ed. by J. Roger Seldin and J. Roger Hindley. Originally circulated as a manuscript in 1969. New York: Academic Press, pp. 375–402.

Milewski, Bartosz (2019). *Category Theory for Programmers*. Serialized online from 2014, print editions published August 12, 2019. Self-published. ISBN: 978-0464183648.

Lane, Saunders Mac (1971). *Categories for the Working Mathematician*. 2nd ed. Vol. 5. Graduate Texts in Mathematics. Second edition, expanded, 1998. Springer Science+Business Media.

Nadal, Pedro Bonilla (Sept. 2021). "Dependent Typing through Closed Cartesian Categories". Trabajo de Fin de Máster. Madrid: Universidad Autónoma de Madrid.

Nagele, Julian, Vincent van Oostrom, and Christian Sternagel (2016). "A Short Mechanized Proof of the Church–Rosser Theorem by the Z-property for the $lambda-beta$-calculus in Nominal Isabelle". In: *arXiv preprint arXiv:1609.03139*. arXiv: `1609.03139 [cs.LO]`. URL: `https://arxiv.org/abs/1609.03139`.

Selinger, Peter (2008). *Lecture Notes on the Lambda Calculus*. Lecture notes / arXiv preprint (revised versions available). Developed from courses taught 2001–2013. URL: `https://www.mathstat.dal.ca/~selinger/papers/lambdanotes.pdf`.

Nederpelt, Rob and Herman Geuvers (2014). *Type Theory and Formal Proof: An Introduction*. Cambridge, UK: Cambridge University Press. ISBN: 9781107036505.

Project, Open Logic (2025). *Parallel beta reduction and the Church Rosser Property*. Revision 91df0ce. URL: `https://builds.openlogicproject.org/content/lambda-calculus/church-rosser/parallel-beta-reduction.pdf`.

Barendregt, Henk P. and Erik Barendsen (2000). *Introduction to Lambda Calculus*. Available as PDF. Revised edition (Dec. 1998; Mar. 2000). Lecture notes. URL: `https://www.cse.chalmers.se/research/group/logic/TypesSS05/Extra/geuvers.pdf`.

Girard, Jean-Yves (1989). *Proofs and Types*. Trans. by Paul Taylor and Yves Lafont. Cambridge Tracts in Theoretical Computer Science 7. Translated and with appendices by Paul Taylor; typeset/appendix by Yves Lafont. Cambridge, UK: Cambridge University Press. ISBN: 9780521371810.

Nazareth, Dieter and Tobias Nipkow (1996). *Formal Verification of Algorithm W: The Monomorphic Case*. Tech. rep. Accessed: 2025-09-01. Technische Universität München. URL: `https://wwwbroy.in.tum.de/publ/papers/NazarethNipkow_TPiH1996.pdf`.

Barendregt, H. P. (1992). *Lambda Calculi with Types*. Vol. II. Handbook of Logic in Computer Science. Oxford University Press.

Urciuoli, Sebastián, Álvaro Tasistro, and Nora Szasz (2020). "Strong Normalization for the Simply-Typed Lambda Calculus in Constructive Type Theory". In: *Procedia Computer Science* 171. Accessed: 2025-09-01, pp. 123–132. URL: `https://www.sciencedirect.com/science/article/pii/S1571066120300463`.

Sørensen, Morten Heine and Pawe Urzyczyn (2006). *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Amsterdam & Boston: Elsevier / North-Holland. ISBN: 9780444520777.

Ahmed, Amal (2012). *Type Inference for ML*. Accessed: 2025-09-01. URL: `https://www.ccs.neu.edu/home/amal/course/7480-s12/inference-notes.pdf`.

Project, The Open Logic (July 2025). *Intuitionistic Logic*. Revision 91df0ce (2025-07-18). URL: `https://builds.openlogicproject.org/content/intuitionistic-logic/intuitionistic-logic.pdf`.

Pierce, Benjamin C. (1991). *Basic Category Theory for Computer Scientists*. The MIT Press. ISBN: 9780262660716.

Imada, Kei (Jan. 2019). *The Equivalence of Typed Lambda Calculi and Cartesian Closed Categories*. Technical Report. Kei Imada (self-published). URL: `https://keikun555.github.io/documents/lambda.pdf`.

Barr, Michael and Charles Wells (1998). *Category Theory for Computing Science*. Revised and available electronically at `http://www.tac.mta.ca/tac/reprints/articles/22/tr22.pdf`. Les Publications CRM, Montreal.

Jacobs, Bart (1999). *Categorical Logic and Type Theory*. Vol. 141. Studies in Logic and the Foundations of Mathematics. Amsterdam: Elsevier Science, pp. xvii + 760. ISBN: 0-444-50170-3.